IEEE Open Journal of the Communications Society

Received 15 September 2025; revised 10 October 2025; accepted 18 October 2025. Date of publication 24 October 2025; date of current version 3 November 2025.

Digital Object Identifier 10.1109/OJCOMS.2025.3625534

Network Traffic Classification Using Self-Supervised Learning and Confident Learning

EHSAN ESLAMI (Student Member, IEEE), AND WALAA HAMOUDA (Senior Member, IEEE)

Department of Electrical and Computer Engineering, Concordia University, Montreal, QC H3G 1M8, Canada

CORRESPONDING AUTHOR: W. HAMOUDA (e-mail: walaa.hamouda@concordia.ca)

ABSTRACT Network traffic classification (NTC) is vital for efficient network management, security, and performance optimization, particularly with 5G/6G technologies. Traditional methods, such as deep packet inspection (DPI) and port-based identification, struggle with the rise of encrypted traffic and dynamic port allocations. Supervised learning methods provide viable alternatives but rely on large labeled datasets, which are difficult to acquire given the diversity and volume of network traffic. Meanwhile, unsupervised learning methods, while less reliant on labeled data, often exhibit lower accuracy. To address these limitations, we propose a novel framework that first leverages Self-Supervised Learning (SSL) with techniques such as autoencoders (AE) or Tabular Contrastive Learning (TabCL) to generate pseudo-labels from extensive unlabeled datasets, addressing the challenge of limited labeled data. We then apply traffic-adapted Confident Learning (CL) to refine these pseudo-labels, enhancing classification precision by mitigating the impact of noise. Our proposed framework offers a generalizable solution that minimizes the need for extensive labeled data while delivering high accuracy. Extensive simulations and evaluations using three datasets (ISCX VPN-nonVPN, self-generated dataset, and UCDavis–QUIC) demonstrate that our method achieves superior accuracy compared to state-of-the-art techniques in classifying network traffic.

INDEX TERMS Network traffic classification, Internet traffic classification, self-supervised learning, contrastive learning, application identification.

I. INTRODUCTION

NETWORK traffic classification (NTC) is a cornerstone of modern network management and security, enabling the identification of applications and services, such as video-streaming or data traffic applications, based on their traffic patterns [1]. This capability supports essential functions like Quality of Service (QoS) provisioning [2], anomaly detection, and traffic engineering. The exponential growth of Internet traffic [3], driven by the increasing number of connected devices, the expansion of cloud computing, and the rise of Artificial Intelligence (AI)–driven services, highlights the need for effective NTC methods [4]. The deployment of 5G and upcoming 6G networks has further complicated network traffic [5], introducing diverse requirements for network slicing, edge computing, and Ultra-Reliable Low-Latency Communications (URLLC), making robust classification increasingly essential [6]. AI plays a vital role in modern networking, facilitating intelligent traffic management, predictive maintenance, and real-time anomaly detection [7], [8].

For NTC, traditional approaches such as port-based identification and Deep Packet Inspection (DPI) have become inadequate. Port-based methods fail due to dynamic port usage, while DPI struggles with the prevalence of encrypted traffic [9], [10]. As a result, modern supervised and representation-learning approaches now dominate. Machine Learning (ML) and Deep Learning (DL) methods can classify traffic using statistical features [11] and time-series features [12]. Supervised learning trains models on labeled datasets and can achieve state-of-the-art (SOTA) performance when sufficient labels are available. For instance, Support Vector Machines (SVM) and logistic regression remain competitive in Software-Defined Networking (SDN); neural networks report strong accuracy and robustness on common benchmarks; supervised contrastive methods achieve leading results under label imbalance; and adversarially trained

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

models sustain accuracy under perturbations [13], [14], [15], [16]. Unsupervised learning, in contrast, does not require labels and can discover patterns independently [4]. More recently, Self-Supervised Learning (SSL) has shown promise by leveraging abundant unlabeled traffic to learn representations and produce pseudo-labels with a small labeled set, reducing reliance on extensive labeled datasets and improving coverage [17]. For example, contrastive pretraining on flow images achieves strong few-shot and transfer performance; transformer pretraining that tokenizes packet/flow bytes and transfers to downstream NTC tasks; and masked flow-image autoencoding advances SOTA SSL representations [18], [19], [20]. SSL tailored to encrypted protocols likewise reports high accuracy with only a small labeled seed [21].

Despite this progress, important challenges remain. For supervised learning, labeling network traffic is resource-intensive and costly, and labeled datasets often fail to reflect the full diversity of real-world traffic, which reduces model generalizability [22], [23], [24]. Recent analyses show that capture-specific fields (e.g., timestamps/MAC/IP/ports) in common NTC benchmarks raise supervised accuracy; once properly anonymized, accuracy drops sharply, indicating weak generalization [25]. Unsupervised methods, while label-free, often struggle to capture the complex patterns needed for precise traffic classification and typically underperform supervised methods [4]. While SSL offers significant benefits by leveraging unlabeled data, SSL-derived pseudo-labels can contain noise due to imperfect pretext accuracy; if used directly to train the downstream classifier, these errors degrade performance. A principled way to cope with noisy labels is confident learning (CL) [26], which estimates class-conditional label errors from out-of-sample predicted probabilities together with the observed (pseudo) labels and constructs the confident joint—a matrix summarizing how often each observed class appears to belong to each (latent) true class—thus enabling noise-aware filtering or reweighting of pseudo-labels. Bringing CL principles to NTC addresses pseudo-label noise directly and provides a principled bridge from representation learning to reliable training.

To address the aforementioned challenges, this paper proposes a novel framework that combines SSL and CL for NTC. First, our approach uses SSL to generate pseudo-labels for unlabeled traffic flows by pretraining with an unlabeled dataset and fine-tuning with a small labeled dataset. We develop two complementary SSL branches: (i) an autoencoder (AE) reconstructs heterogeneous (continuous/categorical) flow features while softly enforcing algebraic relations among them for more plausible outputs, and (ii) Tabular Contrastive Learning (TabCL) [27] with class-conditioned replacement and dual-head projections. This enhanced TabCL method creates augmented views by replacing features from the same class distribution and projects them through separate heads for continuous and categorical slices to handle feature heterogeneity. These designs ensure that AE focuses on fidelity in reconstruction, while TabCL emphasizes similarity between related views. We then fuse the SSL predictions via a confidence/margin voting rule so that the more decisive branch prevails when the two disagree. Second, inspired by CL, we decouple noise estimation from the final classifier and treat the SSL pseudo-labels as inputs to a CL module. Using k-fold out-of-sample prediction probabilities, we estimate the confident joint and introduce adaptations tailored to network traffic flows: per-class, quantile thresholds (rather than means thresholds) for robustness under skew; a logistic weighting that smoothly down-weights uncertain pseudo-labels instead of discarding data; and a balanced retention constraint that aligns each class's retained training mass with its CL-estimated clean fraction [26]. Third, we train the downstream classifier on the full pseudo-labeled pool using a weighted symmetric cross-entropy objective, where per-sample weights, derived from the CL module via out-of-sample probabilities, down-weight uncertain pseudo-labels while retaining all data.

Although recent SOTA methods attain high accuracy, important gaps persist—namely, reliance on scarce labels in supervised learning, limited precision in unsupervised methods, noise in SSL pseudo-labels, and per-class calibration/imbalance issues. Our framework addresses these gaps by pairing NTC-specific SSL (constraint-aware views, dual-head temperatures, SSL fusion) with NTC-adapted CL (quantile thresholds, logistic weighting, balanced retention) before final training. The contributions of this paper are as follows:

1) We propose the first end-to-end framework integrating SSL and CL for NTC that produces pseudo-labels via AE and TabCL, then denoises them with CL before final training. Our framework incorporates innovative extensions to both components that enhance accuracy and label efficiency beyond standard approaches.
2) We demonstrate the effectiveness of our enhanced AE with constraint-consistent reconstruction and NTC-tailored TabCL with class-conditioned, constraint-preserving views and dual-head projections for generating higher-quality pseudo-labels from unlabeled data than simple AEs or the original TabCL.
3) We show that our traffic-adapted CL improves classification accuracy over the original CL by replacing mean thresholds with per-class quantile thresholds for greater robustness to imbalance and skew, incorporating logistic weights for smoother attenuation of uncertain labels, and balanced retention guided by the confident joint to improve noise robustness.
4) We conduct a thorough evaluation and ablations, including simulations to create a realistic network traffic dataset, and assess our model's performance by comparing it to existing methods, demonstrating superior accuracy and macro-F1 under limited labels.

The rest of this paper is organized as follows: Section II reviews related work in NTC and SSL. Section III formalizes

the problem, assumptions, and notation. Section IV details the proposed framework. Section V presents the simulation and experimental results, and Section VI concludes the paper.

II. RELATED WORK

This section provides a detailed review of the SOTA research, focusing on ML, supervised and unsupervised DL, and SSL methods.

A. TRADITIONAL AND MACHINE LEARNING FOR NTC

Early NTC relied on port-based methods and DPI. Reference [28] proposes BitProb, which encodes flows as bit signatures and uses a probabilistic automaton for protocol assignment. These traditional methods struggle with dynamic ports and encrypted traffic. ML is one of the promising solutions for NTC [13], [29]. For example, [29] utilized supervised ML models such as Decision Tree and Random Forest in an SDN context for NTC. Serag et al. [13] demonstrated classical ML algorithms like SVM and Logistic Regression for SDN traffic classification. These approaches handle encrypted and dynamic traffic well using statistical and flow features, but remain constrained by the high labeling demand of supervised learning.

B. SUPERVISED AND UNSUPERVISED DEEP LEARNING FOR NTC

DL has shown promise in NTC due to its ability to learn complex patterns from raw data [14], [15], [16], [30], [31], [32], [33], [34]. For instance, Lotfollahi et al. [30] proposed "Deep Packet," using Stacked Autoencoders (SAEs) and Convolutional Neural Networks (CNNs) to classify encrypted traffic on the ISCX VPN-nonVPN dataset [12], while reducing manual feature engineering via learned representations. Reference [31] introduced a DL approach combining CNN and Recurrent Neural Networks (RNNs), achieving SOTA performance on the CIC-IOT dataset [35]. In [14], IBNN adds an information-bottleneck layer to a CNN plus a fully connected classifier to retain task-relevant features while discarding noise, achieving 97.6% and 98.2% on ISCX datasets, indicating stronger generalization and robustness. Ma et al. [15] introduce Balanced Supervised Contrastive Learning (BSCL), mapping packets to images for ResNet-50 training with contrastive and imbalance-aware losses; on the ISCX2016, it achieves high traffic-type accuracy. Reference [32] proposes a CNN–Long-Short-Term Memory (LSTM) AE that learns spatiotemporal flow features from statistics and then classifies in latent space, reaching near-perfect accuracy on 1.42M flows (4 classes) while remaining robust to imbalance. Xiao et al.'s RBLJAN [16] is a supervised DL model that processes headers/payloads with joint attention and adversarial training for packet-to-flow classification. On various datasets, RBLJAN reports 98% average F1 and about 97% accuracy. MTEFU [34] uses multi-task supervision (class + bandwidth + duration) with shared CNN, SAE, Gated Recurrent Unit (GRU), and LSTM backbones with early-packet inputs to boost label efficiency, reaching 94.00% class accuracy on QUIC with 150 labels. Despite their success, these supervised DL methods rely heavily on large labeled datasets, which are costly and may not generalize across diverse network environments.

On the other hand, unsupervised learning methods have been explored for NTC to circumvent the need for labeled datasets [36], [37]. Zhang et al. [36] proposed an adversarial training and deep clustering method. Their approach achieved a multi-classification accuracy of 92.2%. Reference [37] propose an unsupervised deep multi-source domain-adaptation NTC model (multi-scale features and alignment) that achieves 89% accuracy. These methods cluster flows by similarity without labels; despite this advantage, they generally lag behind supervised learning in accuracy, particularly in complex and diverse traffic scenarios.

C. SELF-SUPERVISED LEARNING FOR NTC

By learning directly from unlabeled data, SSL methods reduce reliance on large labeled datasets [18], [19], [20], [21], [38], [39]. Towhid and Shahriar [21] applied SSL techniques to encrypted NTC, achieving high accuracy with minimal labeled data across multiple public datasets such as Quick UDP Internet Connection (QUIC) [40] and ISCX VPN-nonVPN datasets. Reference [18] proposes an SSL framework for traffic classification that utilizes the FlowPic method. This approach transforms network flow data into 2D histogram images, which are then processed by contrastive representation learning and CNNs. ET-BERT [19] is a Transformer-based framework for NTC that learns contextualized datagram representations from large unlabeled traffic and then fine-tunes with few labels. It uses Datagram2Token (BURST extraction + byte-bigram tokenization) and two traffic-tailored pretext tasks—Masked BURST Modeling (MBM) and Same-origin BURST Prediction (SBP)—with a 12-layer bidirectional Transformer, supporting packet- and flow-level fine-tuning. ET-BERT delivers strong macro-F1 and accuracy ($> 96\%$) on different datasets. YaTC [20] couples a masked-autoencoder (MAE) plus packet/flow-level Transformer attention over a multi-level flow representation (MFR) byte layout. Pretrained on large unlabeled traffic and fine-tuned with few labels, YaTC achieves SOTA on multiple NTC datasets. These advancements highlight SSL's potential to reduce dependency on labeled data, making it a key focus of our work.

D. COMPARISON WITH EXISTING METHODS

We integrate constraint-aware SSL methods for pseudo-labeling, a traffic-adapted CL module to denoise them, and a weighted Multilayer Perceptron (MLP) for final training. This differs from supervised NTC methods [13], [14], [15], [16], [29], [30], [31], [32], [33], [34], which require extensive labels, by leveraging unlabeled traffic while controlling label noise. Our constraint-consistent reconstruction AE and constraint-aware TabCL with dual-head projections,

combined with traffic-adapted CL extensions, yield better scalability and accuracy than prior SSL baselines [18], [19], [20], [21], [38], [39].

III. PROBLEM DEFINITION AND FORMULATION

We consider NTC over K application classes from heterogeneous flow features. Let $D_s = \{(\mathbf{x}_i, y_i)\}_{i=1}^{|D_s|}$ denote a small labeled dataset with $y_i \in \{1, \ldots, K\}$ and $D_l = \{\mathbf{x}_j\}_{j=1}^{|D_l|}$ a large unlabeled dataset drawn from the same deployment domain. Each flow $\mathbf{x} \in \mathbb{R}^d$ comprises continuous attributes $\mathbf{x}_{\text{cont}}$ (e.g., counts, bytes, durations, rates) and categorical attributes $\mathbf{x}_{\text{cat}}$ (e.g., protocol, direction), encoded numerically. Continuous attributes include statistical features (e.g., total packets, total bytes, flow duration, average packet size) and time-series features (e.g., inter-arrival times, burst patterns). These features are essential for classification because they capture both the aggregate behavior and temporal dynamics of traffic.

Given D_s and D_l, our objective is to learn a multiclass classifier $\mathcal{C}_{\text{final}} : \mathbb{R}^d \to \{1, \ldots, K\}$ that attains high accuracy and macro-F1 under scarce labels by leveraging D_l via self-supervision and training on a noise-aware pseudo-labeled pool. Concretely, we first generate pseudo-labels for D_l through two complementary SSL branches adapted to NTC: (i) an AE with constraint-consistent reconstruction for heterogeneous flow features, and (ii) TabCL with class-conditioned, constraint-preserving augmentations and dual-head projections for feature-type heterogeneity. When both branches are used, their predictions are fused by a simple confidence/margin voting rule. Because such labels may be noisy, we explicitly account for label noise via the CL module tailored to NTC via per-class quantile thresholds, calibration-aware logistic weights, and balanced retention. We then train $\mathcal{C}_{\text{final}}$ on the resulting pool with confidence-aware reweighting.

We adopt a closed-world setting with K known classes. Real-world NTC exhibits class imbalance and calibration drift across classes; pseudo-labels produced by SSL contain label noise. We assume access to flow/header-derived features (no deep packet inspection of payloads). Our focus is robustness to label scarcity and pseudo-label noise. To maintain clarity and consistency, we define the notations used in Sections III–IV in Table 1.

TABLE 1. List of notations.

Symbol	Description
$\mathbf{x}$	Flow feature vector in $\mathbb{R}^d$
$\mathbf{x}_{\text{cont}}$, $\mathbf{x}_{\text{cat}}$	Continuous / categorical feature slices
y, $\tilde{y}$	True label, pseudo-label
D_s	Small labeled dataset $\{(\mathbf{x}_i, y_i)\}$
D_l	Large unlabeled dataset $\{\mathbf{x}_i\}$
K	Number of application classes
d	Feature dimension
$\mathcal{E}$, $\mathcal{D}$, $\mathcal{C}$	Encoder, decoder, classifier
$\widehat{\mathbf{x}}$	Autoencoder reconstruction of $\mathbf{x}$
$\mathcal{L}$, θ	Loss function, model parameters
$\mathcal{G}$	Set of algebraic constraints
g	Residual function
ϕ	Weights for constraint residuals in AE
$\widetilde{\mathbf{x}}_i^{(t)}$	replaced view in TabCL
$\check{\mathbf{x}}_i^{(t)}$	Adjusted view by constraint-projection
r	TabCL replacement rate
$\mathcal{P}$	Projection head (TabCL)
$\mathbf{v}_i^{(t)}$	Contrastive projection vector
τ	Temperature (TabCL)
N	Mini-batch size
λ	Mixing coefficient in final TabCL loss
F	Number of folds for CV
η	Learning rate
$\mathbf{p}$	Predicted probability vector
$\widehat{c}(\mathbf{x})$	fused pseudo-label (final vote)
$m(\mathbf{x})$	Confidence margins (top-1 minus runner-up)
s_i	Self-confidence for sample i
$\mathcal{S}_j$	Self-confidence set for class j
$t_j^{(q)}$, q	q-quantile threshold for class j; quantile level
σ_j	robust scale for class j
γ	slope
w_i	Calibration-aware per-sample weight
$w_{\min}$	minimum weight for CL
$\widehat{Q}$	Confident joint matrix
ρ_j	estimated clean fraction for class j
n_j	Count in class j
T_j, M_j, a_j	Target mass; current mass; class scaling factor
w_i'	Final per-sample weight
$\text{clip}(x, \ell, u)$	Clipping operator $\min(\max(x, \ell), u)$
α, β	SCE loss coefficients (forward/reverse CE)

IV. SELF-SUPERVISED LEARNING FOR PSEUDO-LABELING AND CONFIDENT LEARNING

This section first presents a system-level model and data flow, from raw packet capture to the unlabeled flow dataset and ultimately to classification. We then detail the tailored SSL techniques (AE and TabCL) independently, pseudo-label fusion, and the traffic-adapted CL used to train the final classifier.

A. SYSTEM MODEL AND END-TO-END DATA FLOW

Raw packet traces are passively captured at monitored links. Packets are aggregated into bidirectional flows using the 5-tuple (source IP, destination IP, source port, destination port, protocol) with idle/active timeouts; time windows may segment long connections for memory safety. The captured flows are then preprocessed to extract features. These features can be broadly categorized into statistical [11] and time-series features [12]. Statistical features include metrics such as the total number of packets in a flow, the total bytes transferred, the duration of the flow, and the average packet size. These features provide a summary of the flow's characteristics and can help distinguish between different types of applications. For example, video streaming applications might have larger total bytes transferred compared

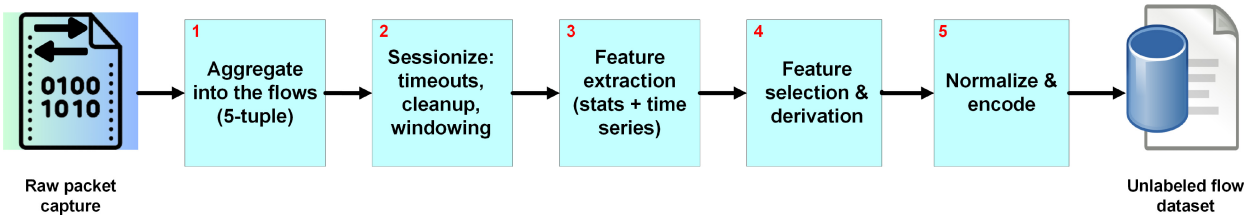


FIGURE 1. Pipeline from raw packet traces to an unlabeled traffic-flow feature dataset.

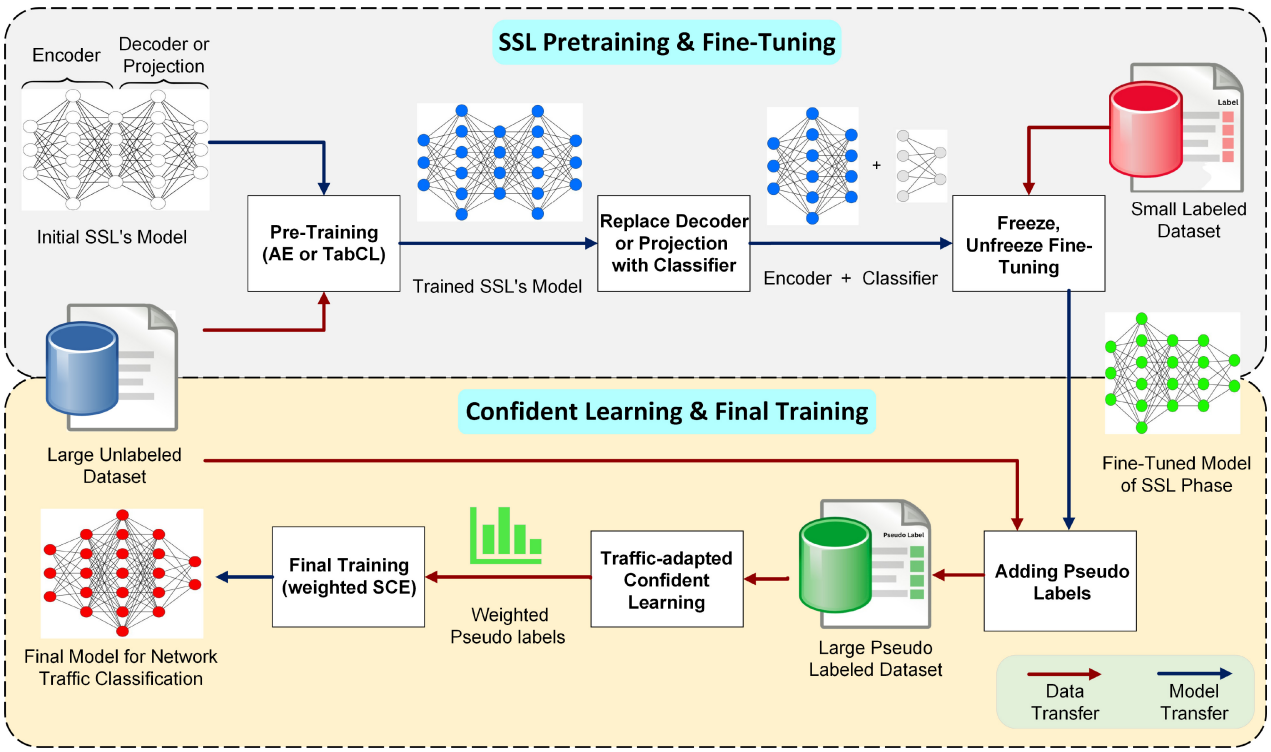


FIGURE 2. Architectural components of traffic classification with SSL and Confident Learning.

to text-based chat applications. Time-series features, on the other hand, capture the temporal patterns in the traffic. These can include the inter-arrival times between packets, the distribution of packet sizes over time, or the presence of bursty behavior. Such features are particularly useful for identifying applications with specific timing requirements, such as real-time communications or periodic updates.

Beyond the primary features, we also derive secondary features by simple aggregations and ratios (e.g., mean/variance of inter-arrival time (IAT), byte/packet rates, up/down throughput, and burst-length ratios), which enrich the tabular representation without accessing payloads. In addition to these numeric attributes, there are categorical features such as protocol and flow direction. The categorical fields are encoded as one-hot or embeddings, while numeric fields are standardized/normalized. This produces a large *unlabeled* traffic-flow dataset D_l; a small *labeled* subset D_s is curated via DPI/heuristics/manual checks and used only for fine-tuning and model selection. The pipeline of unlabeled dataset generation from raw network packets is illustrated in Fig. 1.

As shown in Fig. 2, our framework employs two self-contained SSL paths to generate pseudo-labels from D_l: an AE tailored to tabular flows, which reconstructs heterogeneous continuous/categorical features; and a TabCL that performs contrastive pretraining with constraint-preserving view generation and dual-head projections specialized for continuous vs. categorical slices. After SSL pretraining, each branch replaces its decoder/projection head with a classifier and is fine-tuned on the small labeled set D_s (freeze–unfreeze). Each branch can independently produce pseudo-labels via its fine-tuned classifier. Using two SSL methods is advantageous because the AE branch is reconstruction-driven and encourages feature-wise consistency and simple algebraic relations, whereas the TabCL branch is discrimination-driven and emphasizes class-consistent invariances while respecting heterogeneity through its dual heads, potentially capturing complementary traffic patterns. When both are available, their predictions can be combined using a confidence-margin voting rule to enhance pseudo-label quality; alternatively, the single-branch

output is used. Following SSL, a traffic-adapted CL module estimates label noise on the pseudo-labeled pool, regardless of whether it came from AE, TabCL, or their fusion. It uses out-of-sample probabilities, builds the confident joint, and assigns per-sample training weights via per-class quantile thresholds with calibration-aware smoothing and balanced retention. The final classifier $\mathcal{C}_{\text{final}}$ is trained on the weighted pseudo-labeled set using symmetric cross-entropy loss. This overview sets the stage; detailed descriptions, algorithms, and mathematical formulas for NTC-aware AE, TabCL, fusion, and CL follow in the next separate subsections.

B. AUTOENCODER-BASED SELF-SUPERVISED LEARNING

An AE learns a compressed representation of the input data by encoding it into a lower-dimensional latent space and reconstructing it, capturing underlying patterns in an unsupervised manner. We represent each network flow as a feature vector $\mathbf{x} \in \mathbb{R}^d$ comprising two types of attributes that are typical of network-traffic data: (i) continuous, numeric features (e.g., counts, durations, or rates) denoted by $\mathbf{x}_{\text{cont}}$, and (ii) categorical features (e.g., protocol, flow direction) denoted collectively by $\mathbf{x}_{\text{cat}}$. Categorical attributes are encoded in standard numeric form (e.g., one-hot or embeddings) and are reconstructed with a cross-entropy objective. This heterogeneous structure is characteristic of tabular network-flow representations and motivates losses that respect both numeric reconstruction fidelity and categorical consistency. We utilize an AE to first learn latent representations from D_l and second, generate pseudo-labels, subsequently fine-tuning with D_s. This AE branch is self-contained and can be used on its own to produce the pseudo-labeled pool for the downstream CL stage.

1) PRETRAINING OBJECTIVE WITH CONSTRAINT-CONSISTENT RECONSTRUCTION

Let $\mathcal{E}(\mathbf{x};\theta_{\mathcal{E}})$ be the encoder and $\mathcal{D}(\mathbf{h},;\theta_{\mathcal{D}})$ the decoder. Given an input $\mathbf{x}$, the model produces a latent code $\mathbf{h} = \mathcal{E}(\mathbf{x})$ and a reconstruction $\widehat{\mathbf{x}} = \mathcal{D}(\mathbf{h})$, with corresponding splits $\widehat{\mathbf{x}}_{\text{cont}}$ and $\widehat{\mathbf{x}}_{\text{cat}}$. The pretraining loss function combines a mixed reconstruction term with a small, domain-consistent penalty that softly enforces simple algebraic relations among selected continuous coordinates:

$$\mathcal{L}_{\text{AE}} = \underbrace{\text{MSE}\big(\mathbf{x}_{\text{cont}}, \widehat{\mathbf{x}}_{\text{cont}}\big) + \text{CE}_{\text{cat}}\big(\mathbf{x}_{\text{cat}}, \widehat{\mathbf{x}}_{\text{cat}}\big)}_{\text{mixed reconstruction}} + \underbrace{\mathcal{L}_{\text{cons}}}_{\substack{\text{constraint}\\ \text{consistency}}}, \tag{1}$$

where MSE is the mean squared error on continuous features, and CE denotes the sum of cross-entropy terms over all categorical fields (each field modeled with an appropriate softmax). In network-traffic data, several continuous attributes are related by simple algebra (e.g., a rate-like feature equals a bytes-like feature over a duration-like feature; a packet-length-like feature equals the sum of payload- and header-like features). When the AE reconstructs such tuples, encouraging these relations to hold improves plausibility and stabilizes the latent representation. We formalize this with a set of algebraic *residuals* applied to the reconstructed continuous vector $\widehat{\mathbf{x}}_{\text{cont}}$:

$$\mathcal{G} = \big\{\, g_m : \mathbb{R}^{d_{\text{cont}}} \to \mathbb{R} \;\mid\; m = 1, \ldots, |\mathcal{G}| \,\big\},$$

where each $g_m(\widehat{\mathbf{x}}_{\text{cont}})$ is the residual of constraint m when evaluated on the reconstructed continuous vector. We then define

$$\mathcal{L}_{\text{cons}} = \sum_{m=1}^{|\mathcal{G}|} \phi_m \,\big\|\, g_m\big(\widehat{\mathbf{x}}_{\text{cont}}\big) \,\big\|_1, \tag{2}$$

where $\phi_m > 0$ are modest weights. This penalty nudges the encoder and decoder to reconstruct plausible tuples without prescribing any particular feature identity, and improves the semantic quality of the representation reused at fine-tuning time.

2) FINE-TUNING AND PSEUDO-LABELING

After pretraining on D_l with (1), the decoder is replaced by a classifier $\mathcal{C}(\,\cdot\,;\theta_{\mathcal{C}})$ attached to the encoder. We adopt a short freeze–unfreeze schedule. This schedule is a standard practice that reduces optimization drift when transplanting a new head on a pretrained encoder. First, we freeze $\theta_{\mathcal{E}}$ and optimize $\theta_{\mathcal{C}}$ on D_s to stabilize the new head, then jointly unfreeze $(\theta_{\mathcal{E}}, \theta_{\mathcal{C}})$ and fine-tune on D_s with the standard cross-entropy:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|D_s|} \sum_{(\mathbf{x},y)\in D_s} \log\Big(\mathcal{C}\big(\mathcal{E}(\mathbf{x})\big)[y]\Big). \tag{3}$$

After fine-tuning, we apply the trained head to all $\mathbf{x} \in D_l$ and assign pseudo-labels by top-1 prediction.

3) HYPERPARAMETER SELECTION

Latent width, learning rates, the freeze duration, and the coefficients $\{\phi_m\}$ in (2) are chosen by stratified cross-validation on D_s, targeting macro-F1 after fine-tuning. This keeps the added constraint term modest and data-adaptive, avoiding over-regularization on classes whose statistics naturally vary. Algorithm 1 summarizes the procedure.

C. TABULAR CONTRASTIVE LEARNING (TABCL)

TabCL offers an alternative SSL approach by learning invariant representations through contrastive loss, enhancing the model's ability to generalize across diverse traffic flows. TabCL leverages contrastive learning to maximize the similarity between augmented versions of the same data sample (positive pairs) while distinguishing it from others (negative pairs), making it particularly effective for tabular data. The detailed workflow of TabCL method is presented in Fig. 3. We adopt the standard two-view NT-Xent setup and tailor it to network-traffic data with two components: a class-conditioned replacement scheme to create views, and a constraint-preserving projection that enforces simple identities among reconstructed continuous attributes in

Algorithm 1 Autoencoder-SSL With Constraint-Consistent Reconstruction

Require: Unlabeled D_l, labeled D_s, batch size m, learning rate η

Initialize encoder $\mathcal{E}(\cdot;\theta_{\mathcal{E}})$, decoder $\mathcal{D}(\cdot;\theta_{\mathcal{D}})$

Pretrain on D_l:

for epoch $= 1, 2, \ldots$ **do**

  for mini-batch $\{\mathbf{x}_i\}_{i=1}^{m} \subset D_l$ **do**

    $\mathbf{z}_i \leftarrow \mathcal{E}(\mathbf{x}_i)$; $\widehat{\mathbf{x}}_i \leftarrow \mathcal{D}(\mathbf{z}_i)$

    Compute $\mathcal{L}_{\text{AE}}$ from (1) with $\mathcal{L}_{\text{cons}}$ in (2) (weights ϕ_m); update $(\theta_{\mathcal{E}}, \theta_{\mathcal{D}}) \leftarrow (\theta_{\mathcal{E}}, \theta_{\mathcal{D}}) - \eta \nabla \mathcal{L}_{\text{AE}}$.

Replace $\mathcal{D}$ by classifier $\mathcal{C}(\cdot;\theta_{\mathcal{C}})$

Freeze $\theta_{\mathcal{E}}$; train $\theta_{\mathcal{C}}$ on D_s using CE (3)

Unfreeze $(\theta_{\mathcal{E}}, \theta_{\mathcal{C}})$; joint fine-tuning on D_s with (3)

Pseudo-labeling: for all $\mathbf{x} \in D_l$, set $\widehat{c} \leftarrow \arg\max_k \mathcal{C}(\mathcal{E}(\mathbf{x}))[k]$

Ensure: Pseudo-labeled $D_l = \{(\mathbf{x}, \widehat{c})\}$

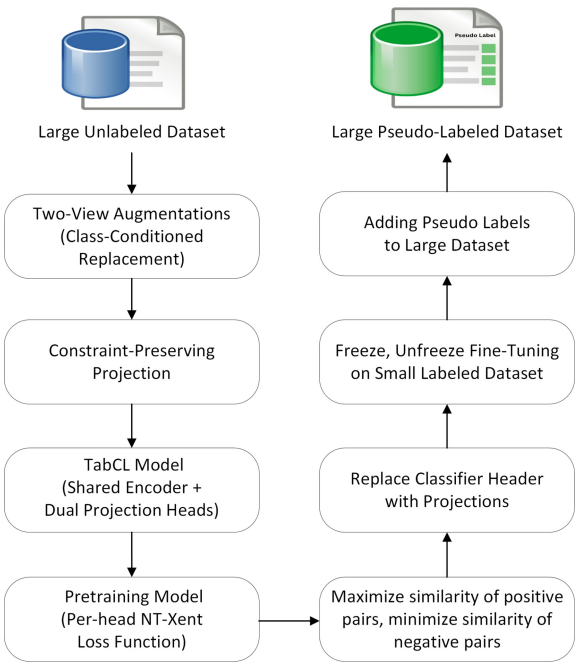


FIGURE 3. Workflow of TabCL for pseudo-labeling.

network traffic datasets. In addition, we use a dual-head contrastive objective to accommodate heterogeneous feature types with distinct temperature scales. This TabCL branch is independent of AE and can be run alone to generate the pseudo-labeled pool.

1) TWO-VIEW AUGMENTATIONS FOR TABULAR FLOWS

Before pretraining in TabCL, we first obtain primary pseudo-labels $\{\tilde{y}_i^{(0)}\}$ for D_l by training a light classifier on D_s and applying it to D_l. During contrastive pretraining, we periodically refresh these pseudo-labels: at predefined intervals, we freeze the current encoder $\mathcal{E}$, train a fresh linear probe on $\{\mathcal{E}(\mathbf{x}), y\}_{(\mathbf{x},y)\in D_s}$, and reclassify $\{\mathcal{E}(\mathbf{x})\}_{\mathbf{x}\in D_l}$ to obtain $\{\tilde{y}_i^{(t)}\}$. Refreshing stops after a small number of rounds or once the fraction of changed labels falls below a tolerance.

For each anchor $\mathbf{x}_i$ with current class identity $c_i = \tilde{y}_i^{(t)}$, we create two views $\widetilde{\mathbf{x}}_i^{(1)}$ and $\widetilde{\mathbf{x}}_i^{(2)}$ by replacing a small subset of the anchor's features. Let d denote the number of features (coordinates) of $\mathbf{x}_i$, and let $r \in (0, 1)$ be the replacement rate. In each view, we randomly select exactly $\lceil rd \rceil$ features of the anchor to replace. Denote by $\mathcal{I}^{(t)} \subset \{1, \ldots, d\}$ the index set chosen for view $t \in \{1, 2\}$; the two sets are sampled independently (preferably with minimal overlap). For every index $j \in \mathcal{I}^{(t)}$, the new value is drawn from the empirical distribution of the same feature among flows currently labeled c_i, while all other features are kept equal to the anchor:

$$\widetilde{\mathbf{x}}_i^{(t)}[j] \sim \mathbb{P}\big(\mathbf{x}[j] \mid \tilde{y} = c_i\big), j \in \mathcal{I}^{(t)}, \tag{4}$$

$$\widetilde{\mathbf{x}}_i^{(t)}[k] = \mathbf{x}_i[k], k \notin \mathcal{I}^{(t)}. \tag{5}$$

This class-conditioned replacement (TabCL principle) keeps views semantically close to the anchor $\mathbf{x}_i$ while inducing invariances within the class manifold.

After replacement, we update each view to satisfy simple algebraic identities among continuous features, using the residual set $\mathcal{G}$ defined in the AE subsection. In practice, we adjust the augmented vector only as much as needed to restore identities (e.g., "rate = bytes / duration" or "packet length = payload + header"), keeping the view close to the original replacement outcome while making the tuple plausible in terms of inter-feature consistency. Let $\breve{\mathbf{x}}_i^{(t)}$ denote the adjusted view:

$$\begin{aligned} \breve{\mathbf{x}}_i^{(t)} &= \Pi_{\mathcal{G}}\big(\widetilde{\mathbf{x}}_i^{(t)}\big), \\ &\triangleq \arg\min_{\mathbf{u}} \ \big\|\mathbf{u} - \widetilde{\mathbf{x}}_i^{(t)}\big\|_2^2 \\ &\quad \text{s.t. } g_m\big(\mathbf{u}_{\text{cont}}\big) = 0, \ \forall g_m \in \mathcal{G}. \end{aligned} \tag{6}$$

Here, $\mathbf{u}$ is the optimization variable (the adjusted vector we solve for), $\mathbf{u}_{\text{cont}}$ is its continuous slice, $\Pi_{\mathcal{G}}(.)$ denotes the projection operator that returns the nearest vector that satisfies all constraints in $\mathcal{G}$, and arg min selects the point that minimizes the squared distance to the replaced view. This constraint-preserving update is simple (closed-form or single-pass updates for ratio/product/sum identities) and ensures that positives are plausible flow tuples. Although applied at the view level, the induced gradients during pretraining bias the encoder's representation toward respecting such identities, which we later reuse for supervised fine-tuning.

2) DUAL-HEAD PROJECTIONS FOR HETEROGENEOUS FEATURES

After creating two augmented views by class-conditioned replacement and constraint-preserving update, in the pretraining phase, we adopt the Normalized Temperature-scaled Cross-Entropy (NT-Xent) loss, which promotes similarity

between augmented views. For projection vectors $\mathbf{v}_i$, $\mathbf{v}_j$ and temperature $\tau > 0$, the NT-Xent term is:

$$\mathcal{L}_{\text{NT-Xent}}(\mathbf{v}_i, \mathbf{v}_j, \tau) = -\log\left(\frac{\exp\big(\text{sim}(\mathbf{v}_i, \mathbf{v}_j)/\tau\big)}{\sum_{k\neq i}\exp(\text{sim}(\mathbf{v}_i, \mathbf{v}_k)/\tau)}\right). \quad (7)$$

Here, $\text{sim}(\mathbf{v}_i, \mathbf{v}_j)$ denotes the cosine similarity between the projection vectors:

$$\text{sim}(\mathbf{v}_i, \mathbf{v}_j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{|\mathbf{v}_i||\mathbf{v}_j|}. \quad (8)$$

For each mini-batch containing N anchors, we form two augmented views per anchor and each view is encoded by the shared backbone and then mapped by two independent projection heads tailored to the feature types: we compute the embedding $\mathbf{z}_i^{(t)} = \mathcal{E}(\check{\mathbf{x}}_i^{(t)})$, feed it to a continuous-focused projector $\mathcal{P}_{\text{cont}}$ to obtain $\mathbf{v}_{\text{cont},i}^{(t)} = \mathcal{P}_{\text{cont}}(\mathbf{z}_i^{(t)})$, and to a categorical-focused projector $\mathcal{P}_{\text{cat}}$ to obtain $\mathbf{v}_{\text{cat},i}^{(t)} = \mathcal{P}_{\text{cat}}(\mathbf{z}_i^{(t)})$ for $t \in \{1, 2\}$ and $i = 1, \ldots, N$. Each anchor, therefore, appears twice (once per view) as the "query," and for each query we use its counterpart view as the positive and all other in-batch projected views from the same head as negatives. To preserve symmetry, we evaluate NT-Xent in both directions (view 1 as anchor with view 2 as positive, and vice versa) and average across the resulting $2N$ anchorings per head. The head-specific losses are thus

$$\mathcal{L}_{\text{cont}} = \frac{1}{2N}\sum_{i=1}^{N}\Big[\mathcal{L}_{\text{NT-Xent}}(\mathbf{v}_{\text{cont},i}^{(1)}, \mathbf{v}_{\text{cont},i}^{(2)}, \tau_{\text{cont}}) + \mathcal{L}_{\text{NT-Xent}}(\mathbf{v}_{\text{cont},i}^{(2)}, \mathbf{v}_{\text{cont},i}^{(1)}, \tau_{\text{cont}})\Big]. \quad (9)$$

$$\mathcal{L}_{\text{cat}} = \frac{1}{2N}\sum_{i=1}^{N}\Big[\mathcal{L}_{\text{NT-Xent}}(\mathbf{v}_{\text{cat},i}^{(1)}, \mathbf{v}_{\text{cat},i}^{(2)}, \tau_{\text{cat}}) + \mathcal{L}_{\text{NT-Xent}}(\mathbf{v}_{\text{cat},i}^{(2)}, \mathbf{v}_{\text{cat},i}^{(1)}, \tau_{\text{cat}})\Big]. \quad (10)$$

The overall contrastive objective combines the two heads with a convex weight:

$$\mathcal{L}_{\text{TabCL}} = \lambda\,\mathcal{L}_{\text{cont}} + (1-\lambda)\,\mathcal{L}_{\text{cat}}, \qquad \lambda \in [0, 1]. \quad (11)$$

In our framework, instead of a single projection head with a single temperature shared across all features, we employ dual-head projections with head-specific temperatures. This design acknowledges that network-flow tables are heterogeneous features.

Using distinct temperatures is beneficial because categorical similarities are typically sharper (best handled with a smaller τ_{cat}), while continuous similarities are more variable (more stable with a larger τ_{cont}). The shared encoder thus learns representations that are simultaneously robust for both feature types, while each head specializes its geometry (via temperature) to the statistics of its slice.

3) FINE-TUNING, PSEUDO-LABELING, AND HYPERPARAMETERS

After contrastive pretraining, we discard $\mathcal{P}_{\text{cont}}$ and $\mathcal{P}_{\text{cat}}$, attach a classifier $\mathcal{C}(\cdot; \theta_{\mathcal{C}})$ to $\mathcal{E}$, and fine-tune on D_s using the freeze–unfreeze schedule and cross-entropy (cf. Eq. (3)). We then apply $\mathcal{C} \circ \mathcal{E}$ to all $\mathbf{x} \in D_l$ to obtain final pseudo-labels. The hyperparameters such as temperatures $(\tau_{\text{cat}}, \tau_{\text{cont}})$, the mixing coefficient λ, the replacement rate r, and the refresh schedule are selected on D_s via stratified cross-validation to maximize macro-F1 after fine-tuning. Algorithm 2 details the expanded procedure.

Algorithm 2 TabCL With Class-Conditioned Replacement, Constraint Projection, and Dual-Head Objective

Require: Unlabeled D_l, labeled D_s; replacement rate r; refresh interval; temperatures $\tau_{\text{cont}}, \tau_{\text{cat}}$; mixing λ

Bootstrap: train a light classifier on D_s; get initial $\{\tilde{y}_i^{(0)}\}$ for D_l

Initialize encoder $\mathcal{E}$ and projectors $\mathcal{P}_{\text{cont}}, \mathcal{P}_{\text{cat}}$

for epoch $= 1, 2, \ldots$ **do**

  if refresh interval reached **then**

    Freeze $\mathcal{E}$; train a linear probe on $\{(\mathcal{E}(\mathbf{x}), y)\}_{D_s}$; reclassify $\{\mathcal{E}(\mathbf{x})\}_{D_l}$ to obtain $\{\tilde{y}_i^{(t)}\}$; stop refreshing if label-change fraction $<$ tolerance or max rounds reached.

  for mini-batch $\{\mathbf{x}_i\}_{i=1}^{N} \subset D_l$ **do**

    For each i: set $c_i \leftarrow \tilde{y}_i^{(t)}$; sample index sets $\mathcal{I}^{(1)}, \mathcal{I}^{(2)}$ of size $\lceil rd \rceil$

    For $t \in \{1, 2\}$: replace $\tilde{\mathbf{x}}_i^{(t)}[j] \sim \mathbb{P}(\mathbf{x}[j] \mid \tilde{y} = c_i)$ for $j \in \mathcal{I}^{(t)}$, copy others from $\mathbf{x}_i$; then project to constraints $\check{\mathbf{x}}_i^{(t)} \leftarrow \Pi_{\mathcal{G}}(\tilde{\mathbf{x}}_i^{(t)})$ as in (6).

    Encode/project: $\mathbf{z}_i^{(t)} \leftarrow \mathcal{E}(\check{\mathbf{x}}_i^{(t)})$; $\mathbf{v}_{\text{cont},i}^{(t)} \leftarrow \mathcal{P}_{\text{cont}}(\mathbf{z}_i^{(t)})$; $\mathbf{v}_{\text{cat},i}^{(t)} \leftarrow \mathcal{P}_{\text{cat}}(\mathbf{z}_i^{(t)})$

    Compute $\mathcal{L}_{\text{cont}}$ and $\mathcal{L}_{\text{cat}}$ via (9), (10)

    Compute $\mathcal{L}_{\text{TabCL}} = \lambda\mathcal{L}_{\text{cont}} + (1-\lambda)\mathcal{L}_{\text{cat}}$ (11); update parameters

Discard projectors; attach $\mathcal{C}$ to $\mathcal{E}$; fine-tune on D_s with (3); pseudo-label D_l

Ensure: Pseudo-labeled D_l from TabCL

D. FUSION OF AE AND TABCL PSEUDO-LABELS

Either SSL branch (AE or TabCL) can independently be used to produce a pseudo-labeled pool for the subsequent CL stage. When both are available, we adopt a simple voting rule that consolidates their predictions at negligible overhead. For each flow $\mathbf{x}$, let

$$\widehat{c}^{\text{AE}}(\mathbf{x}) = \arg\max_{k\in\{1,\ldots,K\}} p_k^{\text{AE}}(\mathbf{x}), \quad (12)$$

$$\widehat{c}^{\text{TabCL}}(\mathbf{x}) = \arg\max_{k\in\{1,\ldots,K\}} p_k^{\text{TabCL}}(\mathbf{x}), \quad (13)$$

be the top-1 classes predicted by the AE and TabCL classifiers, where $p^{\text{AE}}(\mathbf{x}), p^{\text{TabCL}}(\mathbf{x}) \in [0, 1]^K$ are their respective softmax probability vectors. Define the *confidence* of each branch as its top-1 probability,

$$s^{\text{AE}}(\mathbf{x}) = \max_k p_k^{\text{AE}}(\mathbf{x}), \qquad s^{\text{TabCL}}(\mathbf{x}) = \max_k p_k^{\text{TabCL}}(\mathbf{x}), \quad (14)$$

Algorithm 3 Confidence–Margin Voting Fusion of AE and TabCL

Require: Trained AE and TabCL classifiers; unlabeled D_l
Ensure: Fused pseudo-labels on D_l

for each $\mathbf{x} \in D_l$ **do**
  $\mathbf{p}^{\mathrm{AE}} \leftarrow$ softmax from AE; $\mathbf{p}^{\mathrm{TabCL}} \leftarrow$ softmax from TabCL
  $\widehat{c}^{\mathrm{AE}} \leftarrow \arg\max_k \mathbf{p}^{\mathrm{AE}}[k]$; $\widehat{c}^{\mathrm{TabCL}} \leftarrow \arg\max_k \mathbf{p}^{\mathrm{TabCL}}[k]$ (12)
  if $\widehat{c}^{\mathrm{AE}} = \widehat{c}^{\mathrm{TabCL}}$ **then**
    $\widehat{c} \leftarrow \widehat{c}^{\mathrm{AE}}$
  else
    $s^{\mathrm{AE}} \leftarrow \max_k \mathbf{p}^{\mathrm{AE}}[k]$; $s^{\mathrm{TabCL}} \leftarrow \max_k \mathbf{p}^{\mathrm{TabCL}}[k]$ (14)
    $m^{\mathrm{AE}} \leftarrow s^{\mathrm{AE}} - \max_{k \neq \widehat{c}^{\mathrm{AE}}} \mathbf{p}^{\mathrm{AE}}[k]$; $m^{\mathrm{TabCL}} \leftarrow s^{\mathrm{TabCL}} - \max_{k \neq \widehat{c}^{\mathrm{TabCL}}} \mathbf{p}^{\mathrm{TabCL}}[k]$ (15)
    Decide $\widehat{c}$ by rule (17): prefer higher confidence; if tied, prefer larger margin; fall back deterministically otherwise.
  Assign $\widehat{c}$ to $\mathbf{x}$

and the *margin* (gap between the highest and the second-highest probabilities),

$$m^{\mathrm{AE}}(\mathbf{x}) = s^{\mathrm{AE}}(\mathbf{x}) - \max_{k \neq \widehat{c}^{\mathrm{AE}}} p_k^{\mathrm{AE}}(\mathbf{x}), \tag{15}$$

$$m^{\mathrm{TabCL}}(\mathbf{x}) = s^{\mathrm{TabCL}}(\mathbf{x}) - \max_{k \neq \widehat{c}^{\mathrm{TabCL}}} p_k^{\mathrm{TabCL}}(\mathbf{x}). \tag{16}$$

The fused pseudo-label $\widehat{c}(\mathbf{x})$ is decided by agreement, then by confidence, then by margin:

$$\widehat{c}(\mathbf{x}) = \begin{cases} \widehat{c}^{\mathrm{AE}}(\mathbf{x}) & \text{if } \widehat{c}^{\mathrm{AE}}(\mathbf{x}) = \widehat{c}^{\mathrm{TabCL}}(\mathbf{x}), \\ \widehat{c}^{\mathrm{AE}}(\mathbf{x}) & \text{if } s^{\mathrm{AE}}(\mathbf{x}) > s^{\mathrm{TabCL}}(\mathbf{x}), \\ \widehat{c}^{\mathrm{TabCL}}(\mathbf{x}) & \text{if } s^{\mathrm{TabCL}}(\mathbf{x}) > s^{\mathrm{AE}}(\mathbf{x}), \\ \widehat{c}^{\mathrm{AE}}(\mathbf{x}) & \text{if } s^{\mathrm{AE}}(\mathbf{x}) = s^{\mathrm{TabCL}}(\mathbf{x}) \\ & \text{and } m^{\mathrm{AE}}(\mathbf{x}) \geq m^{\mathrm{TabCL}}(\mathbf{x}) \\ \widehat{c}^{\mathrm{TabCL}}(\mathbf{x}) & \text{otherwise.} \end{cases} \tag{17}$$

This rule favors the model with the higher top-1 confidence; if confidences tie exactly, the larger margin (sharper decision) prevails. In practice, exact ties are exceedingly rare; the last line provides a deterministic fallback. The resulting single pseudo-labeled set is then passed to the downstream task. The process of fusion is detailed in Algorithm 3.

Fusion often improves corner cases where the two branches disagree on difficult classes, with minimal changes to the pipeline. The cost is additional SSL pretraining (both branches), which we report explicitly in the efficiency section, together with parameter counts and epochs for each path.

E. CONFIDENT LEARNING FOR NOISE REDUCTION AND FINAL CLASSIFICATION

The pseudo-labeled pool $D_l = \{(\mathbf{x}_i, \tilde{y}_i)\}_{i=1}^{|D_l|}$ produced by SSL, as detailed in the preceding subsections, inevitably contains noise. Training directly on these labels degrades performance. We aim to reduce the impact of noisy pseudo-labels before training the final classifier. We therefore adopt and extend CL [26] into a pipeline that down-weights noisy pseudo-labels and trains the final classifier in five stages: First, following standard CL practice, we compute out-of-sample prediction probabilities and define each sample's self-confidence. Second, we introduce class-adaptive, quantile-based confidence thresholds with robust dispersion (MAD) to reflect per-class difficulty. Third, we convert confidence into smooth, calibration-aware per-sample logistic weights that attenuate uncertain pseudo-labels while retaining all data. Fourth, leveraging the CL confident joint, we perform a class-wise balanced retention calibration that aligns each class's retained mass with its estimated clean fraction, mitigating imbalance under label noise. Finally, we train the final classifier $\mathcal{C}_{\mathrm{final}}$ using weighted symmetric cross-entropy on the full pseudo-labeled set. Importantly, we do not remove samples; rather, we reduce the contribution of uncertain ones and calibrate class masses to stabilize learning. The complete CL-driven noise mitigation and downstream training procedure is summarized in Algorithm 4.

Algorithm 4 Traffic-Adapted CL With Quantile Thresholds, Logistic Weights, BRC, and Weighted SCE

Require: Pseudo-labeled $D_l = \{(\mathbf{x}_i, \tilde{y}_i)\}$; small labeled D_s; folds F; grids $\mathcal{Q}$ (quantile q), $\mathcal{W}$ ($w_{\min}$), Γ (slope γ)
Ensure: Trained final classifier $\mathcal{C}_{\mathrm{final}}$

Out-of-sample probabilities: obtain $\mathbf{p}_i$ via F-fold CV (18); set $s_i \leftarrow \mathbf{p}_i[\tilde{y}_i]$ (19)
Thresholds/weights per class:
for $q \in \mathcal{Q}$, $w_{\min} \in \mathcal{W}$, $\gamma \in \Gamma$ **do**
  for each class j **do**
    Compute $t_j^{(q)}$ (20), $\tilde{m}_j$ (21), MAD_j (22), σ_j (23)
  for each i **do**
    $z_i \leftarrow (s_i - t_{\tilde{y}_i}^{(q)})/(\gamma\, \sigma_{\tilde{y}_i})$ (24)
    $w_i \leftarrow w_{\min} + (1 - w_{\min}) \cdot \mathrm{sigm}(z_i)$ (26)
  Build confident joint $\widehat{Q}$ (27); get ρ_j (28); compute a_j (29)
  Final weights: $w_i' \leftarrow \mathrm{clip}(a_j\, w_i,\, w_{\min},\, 1)$ (30) for $\tilde{y}_i = j$
  Train $\mathcal{C}_{\mathrm{final}}$ on D_l with weighted SCE (31)
Deployment: fix $(q, w_{\min}, \gamma)$ according to the chosen configuration; recompute $\{w_i'\}$ on the full pseudo-labeled pool and train $\mathcal{C}_{\mathrm{final}}$ on D_l.

1) PREDICTION PROBABILITIES AND SELF-CONFIDENCE

We first obtain out-of-sample predicted probabilities $\mathbf{p}_i$ for each sample $\mathbf{x}_i$ in D_l using F-fold cross-validation with a simple classifier (a shallow MLP), for each sample $\mathbf{x}_i$, use the model trained on the $F-1$ folds that did not include $\mathbf{x}_i$. Denoting the parameters of this model by $\theta^{(-f(i))}$, we

calculate the K-way probability vector:

$$\mathbf{p}_i = \left[P\big(y{=}1 \mid \mathbf{x}_i; \theta^{(-f(i))}\big), \ldots, P\big(y{=}K \mid \mathbf{x}_i; \theta^{(-f(i))}\big)\right], \quad \mathbf{p}_i \in [0,1]^K, \quad \sum_{k=1}^{K} \mathbf{p}_i[k] = 1. \tag{18}$$

In other words, $\mathbf{p}_i$ is $P(y \mid \mathbf{x}_i)$ evaluated for all classes. The top-1 class of this out-of-sample model is $\hat{y}_i^\star = \arg\max_k \mathbf{p}_i[k]$, which may or may not equal the SSL pseudo-label $\tilde{y}_i$. In CL, the quantity of interest is self-confidence s_i, defined as the probability that the out-of-sample model assigns to the observed (pseudo) label of the sample:

$$s_i = \mathbf{p}_i\big[\tilde{y}_i\big] \in [0,1]. \tag{19}$$

Thus, $\mathbf{p}_i$ is a K-dimensional probability vector ($P(y \mid \mathbf{x}_i)$ across all classes), while s_i is a single scalar entry of that vector corresponding to $\tilde{y}_i$. These out-of-sample predictions and the resulting self-confidences are used exclusively to vet pseudo-labels via CL; they do not provide gradients to $\mathcal{C}_{\text{final}}$.

2) PER-CLASS, QUANTILE-CALIBRATED THRESHOLDS

For each observed class $j \in \{1, \ldots, K\}$, collect the self-confidence scores of samples whose pseudo-label equals j: $\mathcal{S}_j = \{ s_i : \tilde{y}_i = j \}$, and $n_j = |\mathcal{S}_j|$. Let the elements of $\mathcal{S}_j$ in non-decreasing order be $s_{j,(1)} \le s_{j,(2)} \le \cdots \le s_{j,(n_j)}$ (order statistics). For a quantile level $q \in (0,1]$, define the per-class q-quantile threshold by the standard order-statistic rule:

$$t_j^{(q)} = \text{Quantile}_q\big(\mathcal{S}_j\big) \triangleq s_{j,\,(k_j(q))}, \quad k_j(q) = \big\lceil q\, n_j \big\rceil, \quad q \in (0,1]. \tag{20}$$

(Equivalently, $t_j^{(q)}$ is the smallest value for which at least a fraction q of the scores in $\mathcal{S}_j$ are $\le t_j^{(q)}$; linear interpolation can be used if a specific quantile convention is desired.)

We treat q as a hyperparameter shared across classes and select it by stratified cross-validation on the small labeled set D_s. This procedure ties the threshold choice directly to downstream performance while preserving the per-class adaptivity of $t_j^{(q)}$.

Canonical CL [26] uses the per-class mean self-confidence as a threshold $t_j^{(\text{mean})} = \bar{S}_j$. In contrast, we set the class threshold to a quantile of $\mathcal{S}_j$, which offers two advantages for network traffic flows: (i) it adapts to class-dependent score distributions that are often skewed due to heterogeneous or encrypted traffic patterns, making the cutoff robust to outliers; and (ii) it provides an explicit strictness control via q: higher q yields a stricter threshold (fewer samples lie above $t_j^{(q)}$ and will later receive large weights), while lower q is more permissive.

3) CALIBRATION-AWARE PER-SAMPLE WEIGHTS

To avoid hard pruning (outright removal of suspected-noisy samples) and to stabilize training, we define a smooth weight for each sample that depends on its distance to the class threshold and on the dispersion of confidences in that class. For each observed class, we define the class median by:

$$\tilde{m}_j = \text{median}\big(\mathcal{S}_j\big). \tag{21}$$

We measure dispersion using the *Median Absolute Deviation (MAD)*:

$$\text{MAD}_j = \text{median}\big\{\, |s - \tilde{m}_j| : s \in \mathcal{S}_j \,\big\}. \tag{22}$$

We set the classwise scale to σ_j and clip it to avoid division by zero:

$$\sigma_j \leftarrow \max\big(\text{MAD}_j, \varepsilon\big), \quad \varepsilon > 0. \tag{23}$$

Using MAD makes the scale estimate robust to outliers and skewed score distributions common in traffic flows. Given the per-class quantile threshold $t_j^{(q)}$ from (20), define for sample i with $\tilde{y}_i = j$:

$$z_i = \frac{s_i - t_{\tilde{y}_i}^{(q)}}{\gamma\, \sigma_{\tilde{y}_i}}, \tag{24}$$

where $\gamma > 0$ controls the slope around the threshold. We map z_i to $(0,1)$ with the logistic function that is:

$$\text{sigm}(z_i) = \frac{1}{1 + e^{-z_i}}. \tag{25}$$

With a minimum weight $w_{\min} \in (0,1)$, the per-sample training weight is:

$$w_i = w_{\min} + (1 - w_{\min}) \cdot \text{sigm}(z_i) \in [\, w_{\min}, 1]. \tag{26}$$

All hyperparameters in (23)–(26) (e.g., $w_{\min}$, γ, ε) are selected via stratified cross-validation on the small labeled set D_s.

The weighting mechanism in (26) behaves in a stable and interpretable manner. By construction $w_i \in [w_{\min}, 1]$; When $s_i \ll t_{\tilde{y}_i}^{(q)}$, the pseudo-label of the flow is likely noisy, the standardized margin $z_i \to -\infty$ and thus $w_i \to w_{\min}$. Conversely, when $s_i \gg t_{\tilde{y}_i}^{(q)}$, the pseudo-label is likely correct, $z_i \to +\infty$ and $w_i \to 1$. At the threshold, if $s_i = t_{\tilde{y}_i}^{(q)}$ then $z_i = 0$ and $w_i = w_{\min} + \frac{1}{2}(1 - w_{\min})$, i.e., the exact midpoint between the minimum and maximum weights.

4) CONFIDENT JOINT AND CLASS-WISE BALANCED RETENTION

In CL, the matrix $\widehat{Q} \in \mathbb{R}^{K \times K}$ summarizes the estimated joint between the latent true class (rows) and the pseudo-label (columns). It is computed directly from out-of-sample probability vectors $\mathbf{p}_i$ using soft counts. Specifically,

$$\widehat{Q}_{y=k,\,\tilde{y}=j} = \sum_{i:\,\tilde{y}_i = j} \mathbf{p}_i[k], \quad k, j \in \{1, \ldots, K\}, \tag{27}$$

where the row index "$y = k$" denotes the latent class inferred from $\mathbf{p}_i$. The diagonal entry $\widehat{Q}_{j,j}$ estimates the clean mass for class j, while the column sum $\sum_k \widehat{Q}_{k,j}$ is the total mass

observed as j. The corresponding estimated clean fraction for observed class j is:

$$\rho_j \;=\; \frac{\widehat{Q}_{j,j}}{\sum_k \widehat{Q}_{k,j}} \;\in\; [0,1]. \tag{28}$$

To mitigate minority erosion under class imbalance, a Balanced Retention Constraint (BRC) is applied after computing per-sample weights w_i. Define $n_j \triangleq \#\{i : \tilde{y}_i = j\}$, the target effective mass $T_j \triangleq \rho_j N_j$, and the current mass $M_j \triangleq \sum_{i:\tilde{y}_i=j} w_i$. The classwise scaling factor is:

$$a_j \;=\; \frac{T_j}{\max\big(\epsilon,\, M_j\big)}, \tag{29}$$

where $\epsilon > 0$ provides numerical stability. The final per-sample weight used in training is then:

$$w_i' \;=\; \mathrm{clip}\,\big(a_j\, w_i,\; w_{\min},\; 1\big), \tag{30}$$

This soft constraint yields $\sum_{i:\tilde{y}_i=j} w_i' \approx T_j$ (within a small tolerance), aligning the retained mass of each observed class with its clean fraction estimated from (27)–(28).

If $M_j < T_j$, then $a_j > 1$ and all weights in class j are uniformly lifted (subject to clipping), preventing under-retention of harder/minority classes; if $M_j > T_j$, then $a_j < 1$ and the class is gently down-scaled to avoid over-retention. Because $w_i \in [w_{\min}, 1]$, the effective mass per class lies in $[w_{\min} n_j,\, n_j]$; the BRC moves it toward $T_j = \rho_j N_j$, thereby preserving minority classes in proportion to their estimated cleanliness. Equation (27) follows the standard confident joint construction in CL [26], whereas the scaling rule in (29) and (30) constitutes the proposed extension that operationalizes class-aware retention during training.

5) FINAL TRAINING WITH WEIGHTED SCE

The downstream classifier $\mathcal{C}_{\text{final}}$ is trained on the pseudo-labeled dataset $D_l = \{(\mathbf{x}_i, \tilde{y}_i)\}$ using the Symmetric Cross-Entropy (SCE) loss, weighted by the final per-sample weight w_i' from (30). For a sample $(\mathbf{x}_i, \tilde{y}_i)$ with predicted probability vector $\mathbf{p}_i = \mathcal{C}_{\text{final}}(\mathbf{x}_i) \in [0,1]^K$, the loss is:

$$\mathcal{L}_{\text{SCE}}^{(i)} \;\triangleq\; w_i' \big[\, \alpha\, \mathrm{CE}(\mathbf{p}_i, \tilde{y}_i) \;+\; \beta\, \mathrm{RCE}(\tilde{y}_i, \mathbf{p}_i) \,\big]. \tag{31}$$

The coefficients $\alpha, \beta > 0$ are hyperparameters and are selected by stratified cross-validation on the small labeled set D_s. We adopt SCE because it combines the standard cross-entropy (encouraging correct predictions) with a reverse cross-entropy term that penalizes overconfident mistakes, improving robustness under residual label noise in pseudo-labeled data. Concretely,

$$\hat{y}_{i,k} = \begin{cases} 1-\epsilon, & k = \tilde{y}_i, \\ \dfrac{\epsilon}{K-1}, & k \neq \tilde{y}_i, \end{cases} \qquad \epsilon \in (0,1). \tag{32}$$

$$\mathrm{CE}(\mathbf{p}_i, \tilde{y}_i) = -\sum_{k=1}^{K} \hat{y}_{i,k}\, \log \mathbf{p}_i[k],$$

$$\mathrm{RCE}(\tilde{y}_i, \mathbf{p}_i) = -\sum_{k=1}^{K} \mathbf{p}_i[k]\, \log \hat{y}_{i,k}. \tag{33}$$

Then, parameters of $\mathcal{C}_{\text{final}}$ are updated by gradient descent:

$$\theta_{\text{final}} \;\leftarrow\; \theta_{\text{final}} \;-\; \eta\, \nabla_{\theta_{\text{final}}} \mathcal{L}_{\text{SCE}}. \tag{34}$$

Building on the mathematical formulations and procedural details outlined above, our innovations make this foundation more suitable for the unique challenges of network traffic. Unlike original CL, which relies on mean-based thresholds that can be sensitive to skewed confidence distributions common in imbalanced and heterogeneous traffic data, our quantile-calibrated thresholds provide a more robust cutoff by adapting to class-specific variation and outliers; this avoids overly strict filtering for hard classes and overly lax filtering for easy ones. Similarly, while the canonical method often uses hard pruning that risks losing valuable samples, our logistic weights enable nuanced attenuation, retaining partial contributions from uncertain instances to preserve information and maintain dataset richness. Furthermore, the balanced retention mechanism addresses class imbalance by scaling masses according to clean estimates, a step not present in the original that helps preserve minority flows. Together, these enhancements make our module more effective for traffic classification, where class skew and variable noise are prevalent. Beyond this intuition, our ablations and experiments across different datasets (Section V) confirm that this traffic-adapted CL outperforms the original approach in terms of overall classification performance.

V. EVALUATION & RESULTS

The evaluation of the proposed NTC model is presented in this section, with the objective of assessing its performance. The aim is to compare the model's effectiveness using key metrics such as precision, F1-score, recall, and classification accuracy.

A. SIMULATION SETTING AND DATASETS

To assess the proposed model, we utilized two distinct datasets: a self-generated dataset treated as unlabeled for SSL and a small labeled dataset derived from the ISCX VPN-nonVPN dataset [12]. We used a self-generated dataset to demonstrate the full, deployment-style pipeline from raw packet capture to flow-level tabular features under realistic label scarcity and class imbalance. In addition to these datasets, we also evaluate on the UCDavis-QUIC [40] dataset to broaden the evaluation and assess generalization to modern encrypted traffic. Below, we describe each dataset in detail, including how it was created, its characteristics, and its role in the evaluation.

Self-Generated Dataset as an Unlabeled: The large unlabeled dataset was created using GNS3 (Global Network Simulator 3), a widely used tool for simulating network environments. In this setup, we configured a simulated Cisco router with NetFlow, a service that collects and monitors IP traffic data as it passes through the router. The router acted as a NetFlow exporter, sending traffic information to a virtual server set up as a NetFlow collector. This virtual server, operating within a VMware environment,

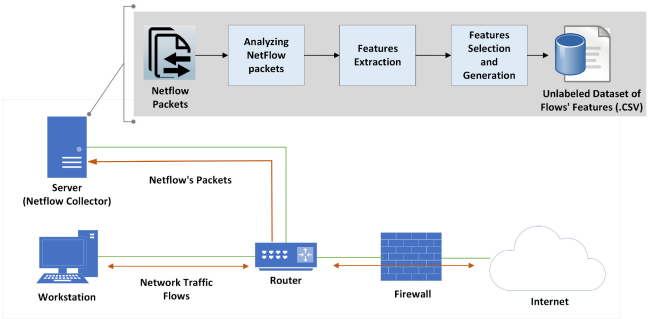


FIGURE 4. Diagram of the network architecture and flowchart of the unlabeled dataset generation using NetFlow.

processed NetFlow packets sent from the router, extracting features from the traffic generated by a Windows 10 virtual machine workstation running 10 distinct applications. These applications were carefully selected to represent different types of network traffic: VPN traffic (e.g., VPN_Vimeo, Tor_YouTube), time-sensitive traffic (e.g., Skype_VideoCall, Facebook_Audio), file transfer traffic (e.g., FTPS_Upload, SFTP_Upload, SCP_Download), and video/voice traffic (e.g., YouTube, Skype_Chat, Email). This variety ensures the dataset covers a broad range of network behaviors seen in real-world scenarios. The combined network architecture and dataset-generation workflow are shown in Fig. 4.

The captured NetFlow data were then processed using a custom Python script that follows this workflow on the NetFlow collector, handling both feature extraction and feature generation. The script extracted statistical features, like packet count, octet count, and throughput, along with time-based attributes such as duration and inter-arrival times. In the "generation" step, the script also processed raw NetFlow packets to calculate additional features (e.g., minimum, maximum, mean, and standard deviation of some features). The final dataset, saved as a Comma-Separated Values (CSV) file, contains 21 features and 8,684 flows. The combined corpus is intentionally imbalanced, reflecting real traffic mixes: YouTube accounts for 41.9% of flows, while minority classes such as SFTP_Upload and SCP_Download are each under 4.5%. This skew is typical in NTC and is handled in our pipeline by class-aware thresholds and balanced retention in the CL stage. Feature distributions are physically plausible and heavy-tailed, as expected for flows: the median Mean Packet Length is 571 B (95th percentile = 1,406 B, below the Ethernet MTU), the median Mean IAT is 6.9 ms (95th percentile = 0.59 s).

ISCX VPN-nonVPN as a Small Labeled Dataset: The smaller labeled dataset was derived from the ISCX VPN-nonVPN dataset [12], a well-known public dataset developed for NTC research. This dataset is originally provided in packet-level as.pcap files, which contain detailed captures of network packets from various activities. It includes a mix of VPN and non-VPN traffic, covering both encrypted and unencrypted flows across different applications and conditions. This diversity makes it a strong benchmark for testing traffic classification models in realistic scenarios, especially where encryption and varying network states are involved. To adapt this dataset for our needs, we used custom Python scripts to process the.pcap files and extract flow-level traffic data. This involved analyzing packet sequences, grouping them into flows based on attributes like source/destination IP addresses and port numbers, and selecting a subset of 538 flows, chosen to limit labeling effort and ensure only complete flows were included. These flows match the same 10 application categories as the large dataset, ensuring consistency. This smaller dataset serves as a labeled resource for fine-tuning the model, despite its much smaller size compared to the self-generated dataset. To reflect real-world traffic distributions, the number of flows per class was intentionally varied, resulting in an imbalanced dataset. This distribution is detailed in Table 2, which enumerates the flow counts for each class.

TABLE 2. Number of flows per class in the self-generated dataset and the ISCX VPN-nonVPN dataset, with class share over the combined corpus.

Class	Number of Flows		Share (%)
	Self gen. Dataset	ISCX Dataset	
YouTube	3764	84	41.85
Skype_VideoCall	438	63	5.44
Skype_Chat	956	41	10.80
FTPS_Upload	638	53	7.48
SFTP_Upload	321	20	3.69
SCP_Download	387	22	4.44
Facebook_Audio	497	59	6.04
Email	904	98	10.86
Tor_YouTube	359	44	4.38
VPN_Vimeo	416	53	5.01
Total	**8684**	**538**	**100.00**

UCDavis-QUIC Dataset: We additionally evaluate on the UCDavis-QUIC dataset, which contains QUIC-only traffic from five Google services (Google Docs, Google Drive, Google Music, Google Search, and YouTube). The released data are packet captures (PCAP) from which flows are derived; in our pipeline, we convert PCAP to CSV and aggregate by 5-tuple to obtain per-flow statistics. Consistent with the original paper of the dataset [40], we employ a flow-level feature schema of 24 numerical statistics per flow. In addition, we retain an explicit categorical flow-direction indicator (forward/backward/bidirectional) alongside the numeric features. The QUIC dataset contains 6,439 flows in total. To mirror our label-scarce setting, we treat only 5% of flows per class as labeled and the remaining 95% as unlabeled, like previous datasets. Table 3 reports the class distribution and the stratified labeled/unlabeled split, making the dataset's imbalance explicit.

We conducted our experiments on a workstation running Ubuntu 22.04.4 LTS, Python 3.12.11, and PyTorch 2.8.0+cu126, equipped with an Intel Xeon CPU @

TABLE 4. Representative grid search results for AE's hyperparameters.

Latent dim	Depth	Hidden width per layer	Dropout	LR	Weight decay	Batch	Epochs	ϕ (constraint)	Macro-F1 (%)
128	3	(256, 128, 64)	0.10	5×10^{-4}	1×10^{-5}	128	100	0.5	**93.8 ± 0.6**
64	2	(256, 128)	0.20	5×10^{-4}	1×10^{-5}	256	100	0.1	92.6 ± 0.6
32	3	(256, 128, 64)	0.00	1×10^{-3}	1×10^{-6}	128	50	1.0	91.2 ± 0.9
64	2	(128, 64)	0.30	1×10^{-3}	1×10^{-4}	64	50	0.0	89.7 ± 0.9
128	3	(256, 128, 64)	0.10	2×10^{-4}	1×10^{-5}	256	80	1.0	92.9 ± 0.5
64	3	(256, 128, 64)	0.20	5×10^{-4}	1×10^{-4}	64	50	2.0	90.1 ± 0.8

TABLE 5. Representative grid search results for TabCL's hyperparameters.

Depth	Hidden widths	LR	Weight decay	Batch	Epochs	τ_{cont}	τ_{cat}	r	λ	Proj dim	Refresh	Macro-F1 (%)
3	(256, 128, 64)	5×10^{-4}	1×10^{-5}	256	100	0.5	0.20	0.15	0.5	128	10	**94.2 ± 0.5**
2	(256, 128)	1×10^{-3}	1×10^{-5}	128	80	0.7	0.20	0.10	0.5	128	5	92.7 ± 0.6
3	(256, 128, 64)	1×10^{-4}	1×10^{-6}	256	100	0.5	0.10	0.10	0.7	256	5	94.0 ± 0.6
2	(128, 64)	5×10^{-4}	1×10^{-4}	512	50	0.5	0.10	0.05	0.4	64	5	91.8 ± 0.8
3	(256, 128, 64)	5×10^{-4}	1×10^{-5}	128	100	0.3	0.05	0.10	0.5	128	10	93.6 ± 0.7
3	(256, 128, 64)	5×10^{-4}	1×10^{-5}	256	100	0.7	0.20	0.20	0.5	128	5	88.3 ± 1.3

TABLE 6. Representative grid search results for hyperparameters of confident learning and final classifier.

CL hyperparameters				Final classifier (MLP, weighted SCE)									Macro-F1 (%)
F-Folds	q	$w_{\min}$	γ	Depth	Hidden widths	LR	Weight decay	Batch	Dropout	Epochs	α	β	
5	0.70	0.20	4	3	(512, 256, 128)	1×10^{-4}	1×10^{-5}	128	0.30	50	0.3	2.0	**95.3 ± 0.5**
5	0.70	0.10	4	3	(512, 256, 128)	5×10^{-4}	1×10^{-5}	128	0.30	80	0.3	2.0	95.0 ± 0.7
3	0.70	0.20	2	2	(256, 128)	5×10^{-4}	1×10^{-5}	256	0.25	30	0.5	1.0	94.6 ± 0.7
5	0.60	0.30	8	3	(1024, 512, 256)	3×10^{-4}	1×10^{-4}	128	0.40	60	0.3	3.0	94.1 ± 0.8
10	0.70	0.20	4	3	(512, 256, 128)	1×10^{-3}	1×10^{-6}	64	0.20	60	0.1	2.0	93.9 ± 0.9
5	0.80	0.10	8	2	(256, 128)	1×10^{-3}	1×10^{-4}	256	0.20	30	0.1	1.0	93.2 ± 0.6

TABLE 3. UCDavis–QUIC class distribution.

Class	Total Flows	Labeled	Unlabeled	Share (%)
Google Docs	1221	61	1160	18.96
Google Drive	1634	82	1552	25.38
Google Music	592	30	562	9.19
Google Search	1915	96	1819	29.74
YouTube	1077	54	1023	16.73
Total	**6439**	**323**	**6116**	**100.00**

2.00 GHz, 12.7 GB RAM, and a single NVIDIA Tesla T4 (15.0 GB) GPU.

B. PERFORMANCE OF THE PROPOSED MODEL ON SELF-GENERATED AND ISCX DATASETS

This subsection evaluates the proposed model's performance on Self-generated and ISCX datasets, focusing on the accuracy of pseudo-label generation via SSL and the final classification accuracy achieved after applying CL to mitigate noisy pseudo-labels. During training, we treated the self-generated dataset as unlabeled, using its assigned labels only for evaluation purposes. By keeping the dataset unlabeled during training, we relied on SSL to uncover patterns from the plentiful unlabeled data. Meanwhile, the small labeled ISCX VPN-nonVPN dataset was used to fine-tune the model. This combination effectively addresses the scarcity of labeled data while leveraging the strengths of both datasets.

The model's performance is measured using the following metrics. In these metrics, TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, respectively.

- *Accuracy:* The proportion of correctly classified samples. This metric provides an overall measure of the model's correctness across all classes, defined as:
$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}. \tag{35}$$
- *Recall:* The ratio of correctly identified positive samples. Recall indicates the model's ability to detect all relevant instances of a class, given by:
$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \tag{36}$$
- *Precision:* The proportion of correctly predicted positive samples among all samples predicted as positive, calculated as:
$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \tag{37}$$
- *F1-Score:* The harmonic mean of precision and recall. The F1-score balances precision and recall, offering a

single metric that reflects the performance of the model, computed as:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{38}$$

The above metrics collectively offer a robust assessment of the model's classification capabilities.

Regarding the hyperparameter selection, we tuned each module with a stratified 5-fold cross-validation (CV) on the small labeled ISCX split; for every candidate configuration, we repeated training with 3 random seeds and reported Macro-F1 as mean ± 95% CI over the resulting 15 scores (5 folds × 3 seeds). We then selected the best settings per module by Macro-F1 and carried those forward to the end-to-end pipeline. Tables 4–6 present representative configurations from the full grid used to select hyperparameters for AE, TabCL, CL, and the final classifier. Finally, Table 7 presents the architecture and hyperparameters for the models utilized in the evaluation section.

1) SELF-SUPERVISED LEARNING FOR PSEUDO-LABEL GENERATION

Two SSL techniques were employed to assign pseudo-labels to the unlabeled dataset: AE and TabCL. These methods were applied separately as pretraining strategies on the unlabeled corpus (8684 flows), followed by fine-tuning with the small labeled set. The pseudo-label accuracy is 87.17% for AE and 89.22% for TabCL. We also evaluated a confidence–margin voting fusion that consolidates AE and TabCL predictions by agreement, which yields 89.91% accuracy—an incremental gain over TabCL alone. In fusion, the improvement is modest because TabCL already provides strong representations and the two models' errors partially overlap; however, fusion slightly increases precision, which is beneficial for the downstream confident-learning filter. Confusion matrices are presented in Fig. 5, illustrating the alignment between pseudo-labels and true labels. Detailed metrics appear in Table 8.

2) FINAL CLASSIFIER WITH CONFIDENT LEARNING

For training the final classifier, we use the fused pseudo-labels produced by the confidence–margin voting of AE and TabCL from the previous subsection (Fig. 5, Table 8). The fused pool attains an overall pseudo-label accuracy of 89.91%, implying that roughly 10% of labels are noisy. If this noise is not addressed and the final multilayer perceptron (MLP) is trained directly on the fused pseudo-labels, the test accuracy plateaus at 90.9%. To improve robustness, we apply traffic-adapted CL to estimate per-sample label reliability and downweight likely errors before final training. With CL, the final MLP reaches an accuracy of **96.29**%. The confusion matrix for the final classifier is shown in Fig. 6, and per-class metrics are reported in Table 9, which shows uniformly high scores across most classes, with the few lower metrics because of feature similarity between these types of traffic. Additionally, a bar chart in Fig. 7 illustrates the accuracy of the final classifier across individual classes, indicating that class-imbalance effects were effectively mitigated by the traffic-adapted CL stage.

Table 10 summarizes a comprehensive ablation across three axes. (i) Pseudo-label source: keeping the downstream CL+MLP stage fixed, we vary the source of pseudo-labels (AE, TabCL, or their fusion). Fusion delivers

TABLE 7. Hyperparameters of models (final choices used in evaluation).

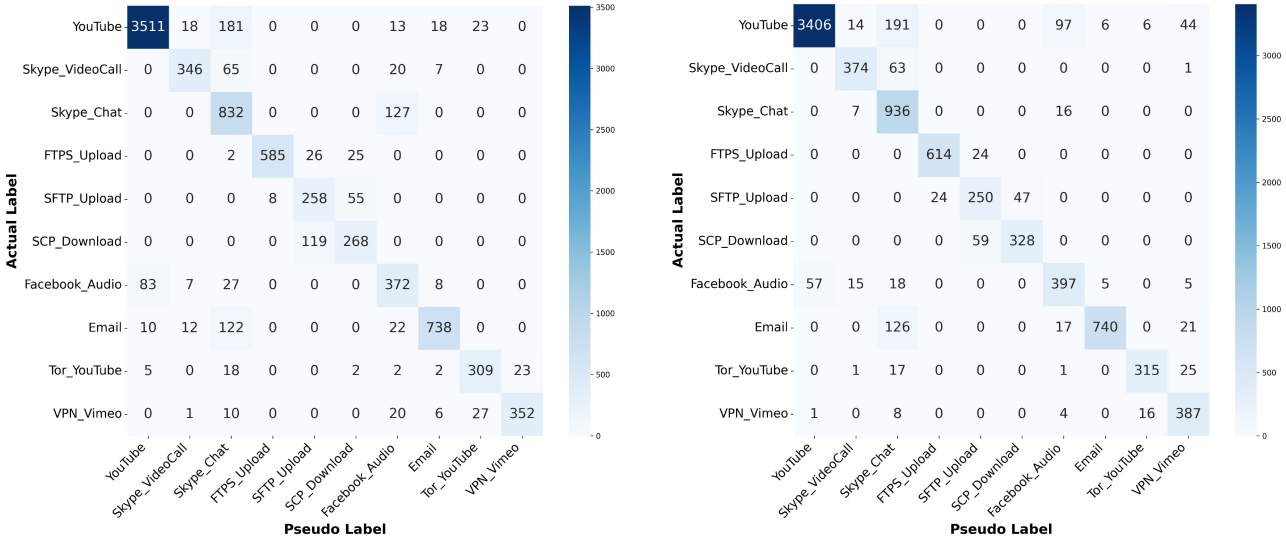
Hyperparameter	Autoencoder	TabCL	Confident Learning	Final Classifier
Model Architecture				
Hidden Layers	3	3	2	3
Neurons in 1st HL	256	256	128	512
Neurons in 2nd HL	128	128	64	256
Neurons in 3rd HL	64	64	–	128
Activation Function	ReLU	ReLU	ReLU	ReLU
Dropout Rate	0.10	0.10	0.20	0.30
Training Parameters				
Optimizer	ADAM	ADAM	ADAM	ADAM
Learning Rate	5×10^{-4}	5×10^{-4}	1×10^{-3}	1×10^{-4}
Batch Size	128	256	64	128
Number of Epochs	100	100	30	50
Weight Decay	1×10^{-5}	1×10^{-5}	–	1×10^{-5}
Specific Parameters				
Latent dimension	128	–	–	–
Constraint weight (ϕ)	0.5	–	–	–
Temperature τ_{cont}	–	0.50	–	–
Temperature τ_{cat}	–	0.20	–	–
Replacement rate (r)	–	0.15	–	–
Head mixing (λ)	–	0.50	–	–
Projection dim (per head)	–	128	–	–
Refresh interval (epochs)	–	10	–	–
F-folds (CL OOS probs)	–	–	5	–
Quantile (q)	–	–	0.70	–
Minimum weight ($w_{\min}$)	–	–	0.20	–
Logistic slope (γ)	–	–	4	–
α (CE term)	–	–	–	0.3
β (RCE term)	–	–	–	2.0

TABLE 8. Performance metrics for SSL pseudo-labeling.

Model	Precision	Recall	F1-score	Accuracy
Autoencoder	88.30 ± 0.3	86.50 ± 0.7	87.20 ± 0.3	87.17 ± 0.3
TabCL	90.10 ± 0.5	88.60 ± 0.4	88.90 ± 0.2	89.22 ± 0.2
(AE ⊕ TabCL)	$\mathbf{90.40 \pm 0.4}$	$\mathbf{89.10 \pm 0.4}$	$\mathbf{89.30 \pm 0.3}$	$\mathbf{89.91 \pm 0.2}$

TABLE 9. Per-class performance of the final classifier.

Class	Precision (%)	Recall (%)	F1 (%)
YouTube	99.23	98.18	98.70
Skype_VideoCall	100.00	96.05	97.99
Skype_Chat	75.19	93.46	83.33
FTPS_Upload	99.45	96.81	98.11
SFTP_Upload	83.81	95.65	89.34
SCP_Download	97.17	90.35	93.64
Facebook_Audio	95.69	91.74	93.67
Email	95.83	94.09	94.95
Tor_YouTube	100.00	92.00	95.83
VPN_Vimeo	98.00	98.99	98.49
Overall (weighted avg)	**96.68**	**96.31**	**96.41**



(a). Autoencoder. (b). TabCL.

FIGURE 5. Confusion matrices of SSL models for adding pseudo-labels: (a) Autoencoder, (b) TabCL.

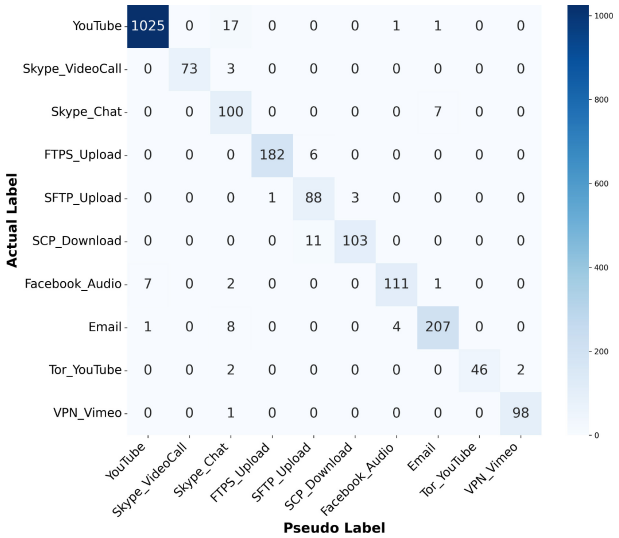


FIGURE 6. Confusion matrix for the MLP final classifier on the test dataset.

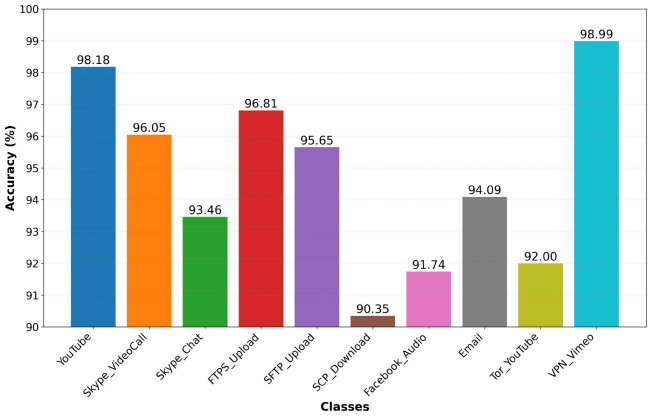


FIGURE 7. Per-class accuracy of final classifier.

consistent gains over either branch alone, indicating that higher-quality, lower-noise pseudo-labels translate directly into stronger downstream performance. (ii) CL variants: reporting CL ablations separately for AE, TabCL, and Fusion pseudo-labels, we progressively enrich CL from a basic mean-threshold filter to our traffic-adapted pipeline (quantile thresholds, logistic weighting, and BRC). Each addition yields measurable improvements, demonstrating that the traffic-aware CL design outperforms the original CL heuristic. (iii) Component contribution: isolating components shows that either SSL (without CL) or CL (without SSL pretraining, just pseudo-labeling with a small labeled dataset) is insufficient; the full pipeline (SSL + CL) is required to reach the best accuracy and macro-F1 with a small labeled dataset.

Here, we report the computational footprint of the two SSL pre-training strategies, the CL method, and the final training. All values are the mean over 10 runs with different random seeds. As summarized in Table 11, the AE is lightweight with 31.19 s per full training round and 1.2 GB peak VRAM, while TabCL trades additional compute for higher-quality representations: 107.86 s per round ($\approx$3.5$\times$ AE) and 1.5 GB peak VRAM. The gap mainly comes from contrastive objectives (multi-view generation and similarity computations). The CL & final MLP stage is the smallest (24.85 s/round, 0.90 GB; 0.018 M parameters) and processes flows fastest (31,240 flows/s), and the pseudo-labeling pass itself is not the bottleneck (AE: 93,464 flows/s; TabCL: 88,409 flows/s), confirming that SSL pretraining, not CL/final training, dominates cost. All stages fit comfortably on a 15 GB T4 and keep peak memory below 2 GB,

TABLE 10. Ablation of the final stage.

Setting	Macro-F1 (%)	Accuracy (%)
Pseudo-label source (for CL+MLP stage)		
AE pseudo-labels + CL	94.3 ± 0.4	94.2 ± 0.4
TabCL pseudo-labels + CL	95.7 ± 0.3	95.6 ± 0.3
Fusion (AE $\oplus$ TabCL) + CL	$\mathbf{96.43 \pm 0.3}$	$\mathbf{96.29 \pm 0.3}$
CL variants (on AE pseudo-labels)		
No-CL (direct training)	88.4 ± 0.6	88.4 ± 0.6
Mean threshold (original CL)	93.0 ± 0.4	92.9 ± 0.4
Quantile threshold only	93.3 ± 0.3	93.2 ± 0.3
+ Logistic weighting	93.8 ± 0.3	93.8 ± 0.2
+ BRC (full CL)	$\mathbf{94.3 \pm 0.4}$	$\mathbf{94.2 \pm 0.4}$
CL variants (on TabCL pseudo-labels)		
No-CL (direct training)	90.1 ± 0.5	90.3 ± 0.5
Mean threshold (original CL)	94.8 ± 0.3	94.8 ± 0.3
Quantile threshold only	95.2 ± 0.3	95.3 ± 0.3
+ Logistic weighting	95.5 ± 0.3	95.5 ± 0.3
+ BRC (full CL)	$\mathbf{95.7 \pm 0.3}$	$\mathbf{95.6 \pm 0.3}$
CL variants (on Fusion pseudo-labels)		
No-CL (direct training)	91.0 ± 0.5	90.9 ± 0.5
Mean threshold (original CL)	95.0 ± 0.4	95.2 ± 0.4
Quantile threshold only	95.4 ± 0.3	95.4 ± 0.3
+ Logistic weighting	96.0 ± 0.2	95.9 ± 0.2
+ BRC (full CL)	$\mathbf{96.43 \pm 0.3}$	$\mathbf{96.29 \pm 0.3}$
Component contribution		
Only SSL (Fusion, no CL)	91.0 ± 0.5	90.9 ± 0.5
Only CL (no SSL pretraining)	85.7 ± 0.6	85.8 ± 0.7
SSL + CL (ours)	$\mathbf{96.43 \pm 0.3}$	$\mathbf{96.29 \pm 0.3}$

TABLE 11. Compute profile of SSL and CL/final training.

Metric	AE	TabCL	CL & Final MLP
Time per round (s)	31.19	107.86	24.85
Training throughput (flows/s)	27,861	8,092	31,240
Pseudo-labeling (flows/s)	93,464	88,409	–
Peak GPU memory (GB)	1.20	1.50	0.90
Average GPU utilization (%)	32.2	43.1	28.7
Trainable parameters (M)	0.032	0.069	0.018

indicating they are deployable on commodity GPUs. In terms of scalability, throughput scales near-linearly with the number of flows, supporting larger datasets. This low overhead enables deployment in real-world scenarios, such as edge networks for online traffic analysis. SSL pretraining can be parallelized across workers or scheduled offline, while the CL & final MLP update remains quick and memory-frugal for frequent refreshes. Overall, the compute profile demonstrates efficiency (minutes per round) and resource frugality for scalable NTC applications.

Fig. 8 visualizes how CL adapts its per-class decision thresholds to the reliability of the pseudo-labels. The dashed gray line is the class mean (original CL), and the

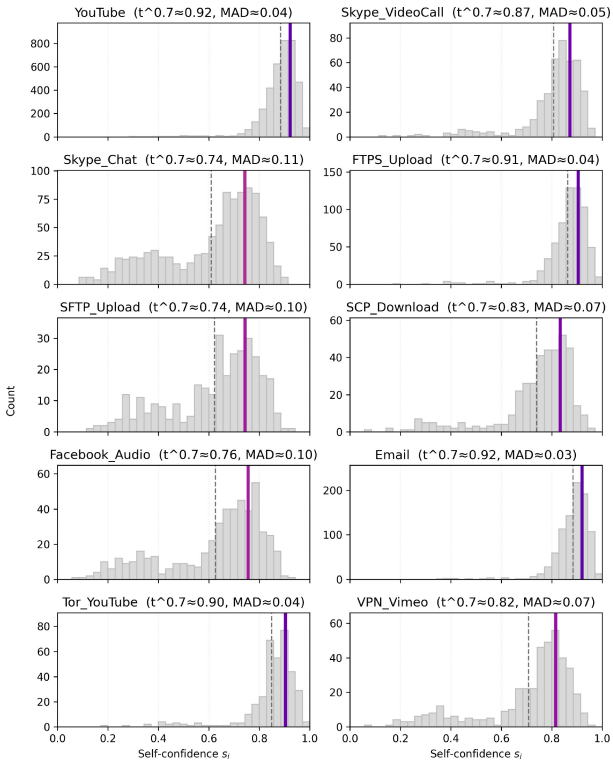


FIGURE 8. Per-class distributions of the self-confidence s_i for all pseudo-labeled flows.

magenta solid line is the quantile threshold $t_j^{0.7}$. Clean classes in the SSL output (e.g., YouTube, FTPS_Upload, and Email) exhibit sharply peaked histograms near $s_i \approx 0.9$–1.0, small MAD (≈ 0.03–0.05), and correspondingly high thresholds $t_j^{0.7} \in [0.90, 0.92]$. In contrast, classes that were noisier in the SSL (e.g., Skype_Chat, SFTP_Upload, and Facebook_Audio) show broader, left-shifted distributions with larger MAD (≈ 0.07–0.11) and noticeably lower thresholds, with $t_j^{0.7}$ moving to 0.74–0.82. This leftward adjustment is intentional: it prevents aggressive pruning of inherently harder classes by requiring a less stringent confidence to contribute significant weight. The dashed gray mean lines are often left of the magenta $t_j^{0.7}$ in clean classes due to right-skew; using the quantile rather than the mean avoids being unduly influenced by a small tail of low-confidence samples. Conversely, in noisy classes, the quantile naturally tracks lower, reflecting the heavier mass at moderate confidences. These adaptive thresholds feed our CL weighting. As a result, the final classifier trained with CL improves from 90.9% (no CL) to 96.29% accuracy. The largest per-class gains occur exactly where Fig. 8 indicates higher noise and lower $t_j^{0.7}$ (e.g., Skype_Chat, SFTP_Upload, and Facebook_Audio), confirming that the performance gap between SSL pseudo-labeling and the final model is attributable to the CL stage.

To further validate our approach, we compare our framework with five recent approaches that span supervised deep models and SSL paradigms: an MTEFU model that combines

TABLE 12. Performance metrics of our model and state-of-the-art methods.

Methods	Authors & Years	Precision	Recall	F1-score	Accuracy
Multitask learning fusion	L. Liu et al. 2024 [34]	90.24	91.2	90.05	92.76
TFE-GNN	H. Zhang et al. 2023 [33]	94.21	93.91	93.88	93.45
SSL FlowPic	E. Horowicz et al. 2024 [18]	92.80	95.22	93.5	93.73
ET-BERT (flow-level)	X. Lin et al. 2022 [19]	93.40	94.01	93.49	91.64
Encrypted Traffic classification with SSL	M. Towhid et al. 2023 [21]	88.4	89.02	87.56	90.56
Our Method	Ours	**96.68**	**96.31**	**96.41**	**96.29**

CNN/SAE/GRU/LSTM heads under a shared architecture (supervised) [34], TFE-GNN (supervised, graph neural model) [33], FlowPic with self-supervised contrastive pre-training on mini-FlowPic images [18], ET-BERT [19] with transformer unsupervised pretraining followed by supervised fine-tuning on limited labels at packet/flow level, and an SSL approach for encrypted traffic [21]. To make the comparison fair and reproducible, we re-implemented (or followed the authors' public descriptions) and evaluated all baselines under our datasets. Specifically, methods that require labels only (TFE-GNN and multitask fusion) were trained on the self-generated dataset and evaluated on the same test partitions as ours; methods that require unlabeled + a small labeled set (FlowPic, ET-BERT, Towhid & Shahriar) were provided our self-generated corpus as unlabeled plus the same small labeled split, mirroring their intended usage.

As summarized in Table 12, our framework achieves 96.29% accuracy while using only a small labeled seed, outperforming FlowPic (93.73%) and ET-BERT (91.64%)under our setting, and exceeding the supervised TFE-GNN and multitask-fusion baselines despite their access to labeled training data. Despite those powerful pre-training strategies, our SSL + CL pipeline delivers higher accuracy with far fewer labels, indicating that flow-tabular representation learning combined with noise-aware CL is competitive with image- and transformer-based pre-training while being label-efficient. Beyond the headline accuracy, our framework operates end-to-end—from raw packet capture to flow aggregation, pseudo-labeling, noise filtering, and final classification—so it can be deployed in realistic "low-label" settings rather than only curated benchmarks. Evaluations on our controlled, end-to-end corpus and on the public ISCX subset indicate that the method scales to thousands of flows and transfers with minimal tuning, suggesting good generalization while directly addressing label scarcity through SSL and CL.

C. PERFORMANCE OF THE PROPOSED MODEL ON UCDAVIS–QUIC

To stress-test generalization beyond our self-generated/ISCX setting, we further evaluate on the UCDavis–QUIC dataset with five Google services (Docs, Drive, Music, Search, YouTube). In keeping with our "low-label" protocol, we treat **5%** of flows per class as the small labeled set and the remaining **95%** as unlabeled (Table 3). Hyperparameters are selected exactly as in the previous subsection (stratified 5-fold CV on the small labeled split; three seeds per candidate; select by Macro-F1). Given the lower number of classes (5) and a more homogeneous feature schema, only minor changes were adopted: based on grid search, for the AE we use a slightly smaller latent size (64) to curb overfitting; in TabCL we reduce augmentation intensity (replacement rate $r = 0.10$) and temperature ($\tau_{\text{cont}} = 0.4$, $\tau_{\text{cat}} = 0.15$) to reflect the smoother intra-class structure; in CL we raise the quantile threshold to $q = 0.75$ and use a steeper logistic slope $\gamma = 6$ to be more selective with residual noise.

TABLE 13. UCDavis–QUIC: SSL pseudo-labeling metrics.

Model	Precision (%)	Recall (%)	F1 (%)	Accuracy (%)
Autoencoder	94.16 ± 0.28	94.07 ± 0.26	94.09 ± 0.24	94.07 ± 0.22
TabCL	95.70 ± 0.22	95.65 ± 0.21	95.66 ± 0.21	95.65 ± 0.20
(AE $\oplus$ TabCL)	$\mathbf{96.20 \pm 0.20}$	$\mathbf{96.10 \pm 0.20}$	$\mathbf{96.15 \pm 0.19}$	$\mathbf{96.11 \pm 0.18}$

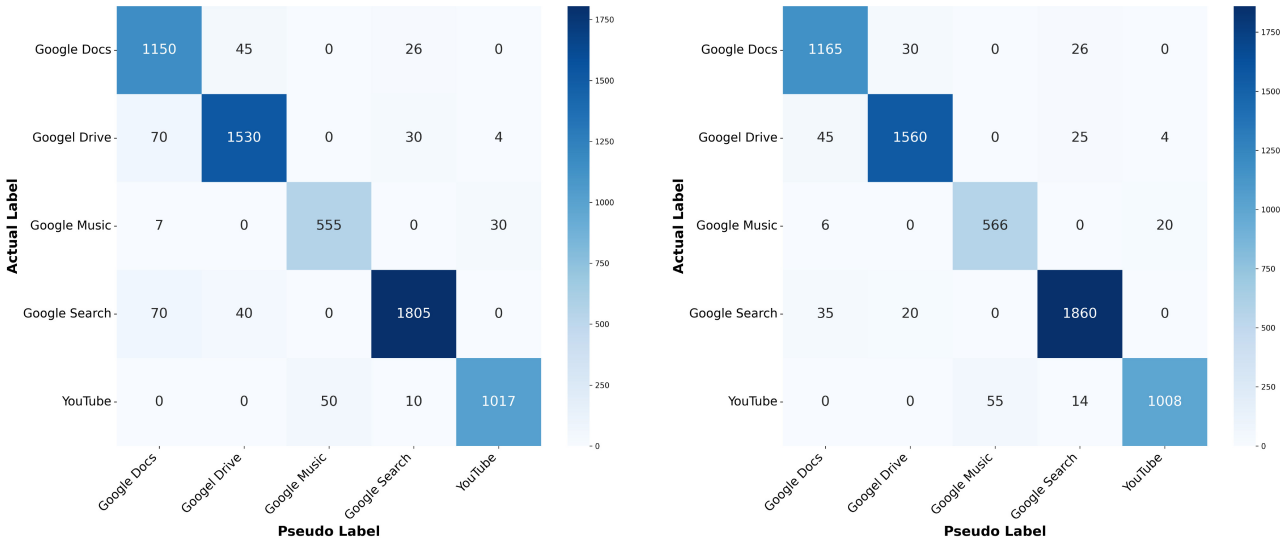
1) SELF-SUPERVISED LEARNING FOR PSEUDO-LABEL GENERATION (QUIC)

We apply both SSL branches independently on the unlabeled QUIC flows and then fine-tune on the 5% labeled split. Pseudo-label performance on the full dataset is strong: the AE reaches 94.07% accuracy, TabCL improves to 95.65%, and a confidence–margin voting fusion gives a further lift to 96.11%. Class-wise confusion patterns for AE and TabCL are shown in Fig. 9. Aggregate metrics of SSL pseudo-labeling are detailed in Table 13.

For completeness, we profiled the SSL round times on the same workstation used earlier. Owing to the smaller corpus (6,439 flows vs. 8,684 previously), both branches are faster: AE averages 22.21 s/round and TabCL 51.95 s/round. Peak GPU memory is modest for both branches—about 1.0 GB for AE and 1.3 GB for TabCL. This is consistent with the roughly linear scaling of throughput with the number of flows observed in the previous subsection.

2) FINAL CLASSIFIER WITH CONFIDENT LEARNING (QUIC)

We then pass the fused pseudo-labeled pool into traffic-adapted CL and train the final MLP. The resulting model attains 98.76% ± 0.10 accuracy. The confusion matrix is shown in Fig. 10. Per-class precision, recall, F1, and



(a). Autoencoder (AE). (b). TabCL.

FIGURE 9. UCDavis–QUIC: confusion matrices of SSL pseudo-labelers.

TABLE 14. UCDavis–QUIC: per-class performance of the final classifier.

Class	Precision (%)	Recall (%)	F1 (%)	Accuracy (%)
Google Docs	97.49	98.77	98.09	98.69
Google Drive	99.20	98.53	98.90	98.59
Google Music	97.67	98.99	98.32	98.99
Google Search	99.22	98.96	99.14	99.06
YouTube	99.25	98.51	98.79	98.33
Overall (weighted)	**98.85**	**98.74**	**98.69**	**98.76**

per-class accuracy are reported in Table 14. Scores are uniformly high; the few residual errors arise primarily from semantically similar services (e.g., Music vs. YouTube, Docs vs. Search).

Table 15 consolidates three ablations using the same protocol as before (mean over 10 seeds). (i) Pseudo-label source. Holding the downstream CL+MLP fixed, (ii) CL variants based on AE, TabCL, and Fusion pseudo-labels. Starting from "No-CL", progressively adding quantile thresholds, logistic weighting, and our BRC step yields monotonic gains, evidence that the traffic-adapted CL is more effective than the original mean-threshold heuristic. (iii) Component contribution Neither SSL (without CL) nor CL (without SSL pretraining) is sufficient; the full SSL+CL pipeline is required to reach the best accuracy/F1 with only 5% labels.

Finally, Table 16 contrasts our method with three representative baselines that report results on UCDavis–QUIC. FlowPic's QUIC "script" setting reports 98.3% accuracy; the multitask fusion model reports $\approx$ 94.7% for "Class" prediction; the SSL method of Towhid & Shahriar reports $\approx$ 98% depending on the pretraining ratio. As seen, our SSL + CL pipeline attains the strongest overall scores on this dataset as well, reinforcing that the framework generalizes across three datasets (in two evaluation settings) and remains competitive even against specialized supervised or SSL baselines.

TABLE 15. Ablation on UCDavis–QUIC.

Setting	Macro-F1 (%)	Accuracy (%)
Pseudo-label source (for CL+MLP)		
AE pseudo-labels + CL	97.33 ± 0.15	97.30 ± 0.15
TabCL pseudo-labels + CL	98.11 ± 0.12	98.05 ± 0.12
Fusion (AE $\oplus$ TabCL) + CL	$\mathbf{98.85 \pm 0.10}$	$\mathbf{98.76 \pm 0.10}$
CL variants (on AE pseudo-labels)		
No-CL (direct training)	94.15 ± 0.22	94.12 ± 0.22
Mean threshold (original CL)	96.80 ± 0.16	96.76 ± 0.16
Quantile threshold only	96.95 ± 0.16	96.90 ± 0.15
+ Logistic weighting	97.17 ± 0.12	97.13 ± 0.13
+ BRC (full CL)	$\mathbf{97.33 \pm 0.15}$	$\mathbf{97.32 \pm 0.15}$
CL variants (on TabCL pseudo-labels)		
No-CL (direct training)	95.75 ± 0.20	95.70 ± 0.20
Mean threshold (original CL)	97.69 ± 0.13	97.64 ± 0.13
Quantile threshold only	97.84 ± 0.14	97.82 ± 0.14
+ Logistic weighting	97.98 ± 0.12	97.89 ± 0.11
+ BRC (full CL)	$\mathbf{98.11 \pm 0.12}$	$\mathbf{98.05 \pm 0.12}$
CL variants (Fusion pseudo-labels)		
No-CL (direct training)	96.22 ± 0.18	96.19 ± 0.18
Mean threshold (original CL)	98.31 ± 0.12	98.25 ± 0.12
Quantile threshold only	98.47 ± 0.12	98.42 ± 0.12
+ Logistic weighting	98.66 ± 0.11	98.62 ± 0.11
+ BRC (full CL)	$\mathbf{98.85 \pm 0.10}$	$\mathbf{98.76 \pm 0.10}$
Component contribution		
Only SSL (Fusion, no CL)	96.24 ± 0.18	96.18 ± 0.18
Only CL (no SSL pretraining)	93.00 ± 0.22	92.90 ± 0.27
SSL + CL (ours)	$\mathbf{98.85 \pm 0.10}$	$\mathbf{98.76 \pm 0.10}$

TABLE 16. UCDavis–QUIC: comparison our model with state-of-the-art methods.

Methods	Authors & Years	Precision	Recall	F1-score	Accuracy
Multitask learning fusion	L. Liu et al., 2024 [34]	94.80	94.58	94.69	94.67
SSL FlowPic (QUIC)	E. Horowicz et al., 2024 [18]	98.76	99.66	98.71	98.30
Encrypted Traffic classification (SSL)	M. Towhid et al., 2023 [21]	98.12	97.93	98.03	98.01
Our Method (SSL + CL)	**Ours**	**98.85**	**98.74**	**98.69**	**98.76**

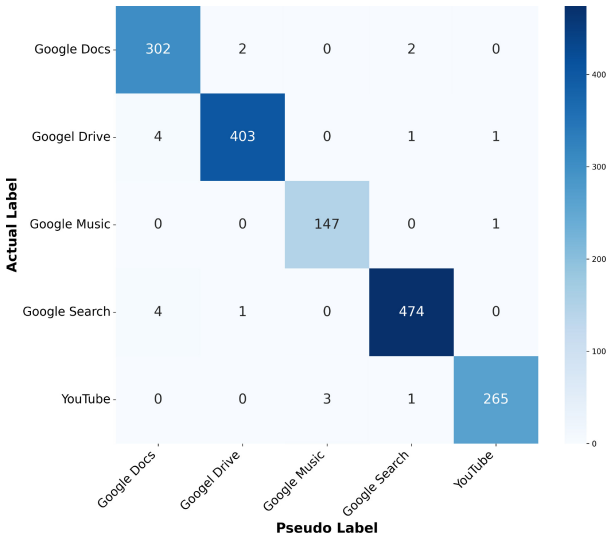


FIGURE 10. UCDavis–QUIC: confusion matrix of the final MLP with CL.

VI. CONCLUSION

This paper presented an end-to-end framework for NTC that couples SSL with a traffic-adapted CL stage and a final weighted MLP classifier. Two complementary SSL branches, an AE with constraint-consistent reconstruction and a TabCL with class-conditioned views and dual-head temperatures, produce high-quality pseudo-labels from unlabeled flows. A confidence–margin voting fusion consolidates the branches, and the downstream CL module denoises pseudo-labels via per-class quantile thresholds, calibration-aware logistic weighting, and balanced retention before final training with symmetric cross-entropy. Across three datasets and two settings, the framework is accurate and label-efficient. On our deployment-style pipeline built from raw packet capture (self-generated unlabeled corpus) and a small ISCX VPN–nonVPN labeled split, the final model attains **96.29%** accuracy with strong macro-F1, while remaining resource-frugal. On the public UCDavis–QUIC dataset, using only **5%** of flows per class as labeled, the method reaches **98.76%** accuracy and uniformly high per-class scores, outperforming strong supervised and SSL baselines reported on the same data. Ablations confirm that (i) higher-quality pseudo-labels (AE/TabCL fusion) translate into better downstream performance, and (ii) the traffic-adapted CL design materially improves over a basic mean-threshold heuristic.

Beyond headline metrics, the framework provides a complete solution from raw packets to flow features, pseudo-labeling, noise mitigation, and classification, supporting realistic "low-label" deployments. Nevertheless, some limitations remain. The current framework relies on offline pretraining, which may limit its applicability in high-throughput real-time systems or when traffic distributions drift quickly. While we evaluated the framework on three datasets in two settings, its performance could be influenced by dataset diversity; for instance, rare enterprise apps and emerging protocols may be under-represented. Additionally, scaling the pipeline to real-time, high-volume traffic with tight latency requirements remains challenging. Moreover, network traffic data are inherently privacy-sensitive; compliance and data-sharing constraints can limit training and cross-site evaluation. To address these, future research could extend the approach to online incremental learning that refreshes representations and class-wise thresholds for dynamic environments, incorporate broader and more diverse datasets to enhance generalization, develop scalable real-time variants, integrate open-world/unknown-class handling, and federated training for privacy-preserving multi-site scenarios.

REFERENCES

[1] A. Azab, M. Khasawneh, S. Alrabaee, K. K. R. Choo, and M. Sarsour, "Network traffic classification: Techniques, datasets, and challenges," *Digit. Commun. Netw.*, vol. 10, no. 3, pp. 676–692, Jun. 2024.

[2] M. Saqib, H. Elbiaze, and R. H. Glitho, "Adaptive in-network traffic classifier: Bridging the gap for improved QoS by minimizing misclassification," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 677–689, 2024.

[3] (Cisco, San Jose, CA, USA). *Cisco Annual Internet Report (2018–2023) White Paper*. Accessed: Mar. 9, 2020. [Online]. Available: https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html.

[4] Y. Feng et al., "Unmasking the Internet: A survey of fine-grained network traffic analysis," *IEEE Commun. Surveys Tuts.*, early access, Feb. 25, 2025, doi: 10.1109/COMST.2025.3545541.

[5] E. Paolini, L. Valcarenghi, L. Maggiani, and N. Andriolli, "Real-time network packet classification exploiting computer vision architectures," *IEEE Open J. Commun. Soc.*, vol. 5 pp. 1155–1166, 2024.

[6] A. Shahraki, M. Abbasi, A. Taherkordi, and A. D. Jurcut, "A comprehensive survey on 6G networks: Applications, core services, enabling technologies, and challenges," Jan. 2021, *arXiv:2101.12475*.

[7] M. Abbasi, A. Shahraki, and A. Taherkordi, "Deep learning for network traffic monitoring and analysis (NTMA): A survey," *Comput. Commun.*, vol. 170, pp. 19–33, Mar. 2021.

[8] D. Di Monda, A. Montieri, V. Persico, P. Voria, M. De Ieso, and A. Pescapè, "Few-shot class-incremental learning for network intrusion detection systems," *IEEE Open J. Commun. Soc.*, vol. 5 pp. 6736–6757, 2024.

[9] P. Khandait, N. Hubballi, and B. Mazumdar, "Efficient keyword matching for deep packet inspection based network traffic classification," in *Proc. IEEE Int. Conf. Commun. Syst. Netw. (COMSNETS)*, Mar. 2020, pp. 567–570.

[10] A. Nascita, G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, "A survey on explainable artificial intelligence for Internet traffic classification and prediction, and intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 5, pp. 3165–3198, 4th Quart., 2025.

[11] R. Raveendran and R. Menon, "An efficient method for Internet traffic classification and identification using statistical features," *Int. J. Eng. Res. Technol.*, vol. 4, no. 7, pp. 297–303, 2015.

[12] G. Draper-Gil, A. Habibi, M. Saiful, and A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2016, pp. 407–414.

[13] R. H. Serag, M. S. Abdalzaher, H. Elsayed, M. Sobh, M. Krichen, and M. Salim, "Machine-learning-based traffic classification in software-defined networks," *Electronics*, vol. 13, no. 6, p. 1108, Mar. 2024.

[14] W. Lin and Y. Chen, "Robust network traffic classification based on information bottleneck neural network," *IEEE Access*, vol. 12, pp. 150169–150179, 2024.

[15] Y. Ma, Z. Li, H. Xue, and J. Chang, "A balanced supervised contrastive learning-based method for encrypted network traffic classification," *Comput. Secur.*, vol. 145, Oct. 2024, Art. no. 104023.

[16] X. Xiao et al., "RBLJAN: Robust byte-label joint attention network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2161–2178, May/Jun. 2025.

[17] K. Lin, X. Xu, and Y. Jiang, "A new semi-supervised approach for network encrypted traffic clustering and classification," in *Proc. IEEE Int. Conf. Comput. Support. Cooper. Work Design (CSCWD)*, May 2022, pp. 41–46.

[18] E. Horowicz, T. Shapira, and Y. Shavitt, "Self-supervised traffic classification: Flow embedding and few-shot solutions," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3054–3067, Jun. 2024.

[19] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.

[20] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, 2023, pp. 5420–5427.

[21] M. Towhid and N. Shahriar, "Encrypted network traffic classification using self-supervised learning," in *Proc. IEEE Int. Conf. Netw. Softw. (NetSoft)*, Aug. 2022, pp. 366–375.

[22] Y. Heng, V. Chandrasekhar, and J. G. Andrews, "UTMobileNetTraffic2021: A labeled public network traffic dataset," *IEEE Netw. Lett.*, vol. 3, no. 3, pp. 156–160, Sep. 2021.

[23] J. L. Guerra, C. Catania, and E. Veas, "Datasets are not enough: Challenges in labeling network traffic," *Comput. Secur.*, vol. 120, Jun. 2022, Art. no. 102810.

[24] D. Soukup, P. Tisovčík, K. Hynek, and T. Čejka, "Towards evaluating quality of datasets for network traffic domain," in *Proc. IEEE Int. Conf. Netw. Service Manag. (CNSM)*, 2021, pp. 264–268.

[25] J. Krupski, M. Iwanowski, and W. Graniszewski, "On the right choice of data from popular datasets for Internet traffic classification," *Comput. Commun. J.*, vol. 233, Mar. 2025, Art. no. 108068.

[26] C. G. Northcutt, L. Jiang, and I. L. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *J. Artif. Intell. Res.*, vol. 70, pp. 1373–1411, Apr. 2021.

[27] W. Cui, R. Hosseinzadeh, J. Ma, T. Wu, Y. Sui, and K. Golestan, "Tabular data contrastive learning via class-conditioned and feature-correlation based augmentation," Apr. 2024, *arXiv:2404.17489*.

[28] N. Hubballi, M. Swarnkar, and M. Conti, "BitProb: Probabilistic Bit Signatures for Accurate Application Identification," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 3, pp. 1730–1741, Sep. 2020.

[29] A. Salau and M. Beyene, "Software defined networking based network traffic classification using machine learning techniques," *Sci. Rep.*, vol. 14, no. 1, Aug. 2024, Art. no. 20060.

[30] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.

[31] A. Jenefa, S. Sam, V. Nair, B. G. Thomas, A. S. George, and R. Thomas, "A robust deep learning-based approach for network traffic classification using CNNs and RNNs," in *Proc. IEEE Int. Conf. Signal Process. Commun. (ICSPC)*, May 2023, pp. 106–110.

[32] G. D'Angelo and F. Palmieri, "Network traffic classification using deep convolutional recurrent autoencoder neural networks for spatial–temporal features extraction," *J. Netw. Comput. Appl.*, vol. 173, Jan. 2021, Art. no. 102890.

[33] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2066–2075.

[34] L. Liu, Y. Yu, Y. Wu, Z. Hui, and J. Hu, "Method for multi-task learning fusion network traffic classification to address small sample labels," *Sci. Rep.*, vol. 14, no. 1, p. 2518, Jan. 2024.

[35] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional IoT profiling dataset," in *Proc. 19th Annu. Int. Conf. Privacy, Security Trust (PST)*, Aug. 2022, pp. 1–11.

[36] W. Zhang, L. Zhang, X. Zhang, Y. Wang, P. Liu, and G. Gui, "Intelligent unsupervised network traffic classification method using adversarial training and deep clustering for secure Internet of Things," *Future Internet*, vol. 15, no. 9, pp. 298–318, 2023.

[37] B. Gao, Y. Yang, Z. Gao, P. Yu, R. Lyu, and S. Chen, "Unsupervised network traffic classification based on multi-source synergistic distribution alignment," in *Proc. IEEE Global Commun. Conf.*, Feb. 2023, pp. 4313–4319.

[38] M. A. Aleisa, "Traffic classification in SDN-based IoT network using two-level fused network with self-adaptive manta ray foraging," *Sci. Rep.*, vol. 15, no. 1, p. 881, Jan. 2025.

[39] S. Xu, J. Han, Y. Liu, H. Liu, and Y. Bai, "Few-shot traffic classification based on autoencoder and deep graph convolutional networks," *Sci. Rep.*, vol. 15, no. 1, p. 8995, Mar. 2025.

[40] S. Rezaei and X. Liu, "How to achieve high classification accuracy with just a few labels: A semi-supervised approach using sampled packets," May 2020, *arXiv:1812.09761*.



EHSAN ESLAMI (Student Member, IEEE) received the B.Sc. degree in computer engineering from the University of Tabriz, Tabriz, Iran, in 2022. He is currently pursuing the M.Sc. degree in computer engineering from Concordia University, Montreal, QC, Canada.

His research interests include AI-powered computer networks, federated learning, programmability of computer networks, and software-defined networking.



WALAA HAMOUDA (Senior Member, IEEE) received the M.A.Sc. and Ph.D. degrees in electrical and computer engineering from Queens University, Kingston, ON, Canada, in 1998 and 2002, respectively. Since July 2002, he has been with the Department of Electrical and Computer Engineering, Concordia University, Montreal, QC, Canada, where he is currently a Professor. Since June 2006, he is Concordia University Research Chair Tier I in Wireless Communications and Networking. His current research interests include machine-to-machine communications, IoT, 5G and beyond technologies, single/multiuser multiple-input–multiple-output communications, space-time processing, cognitive radios, and wireless networks. He received numerous awards, including the Best Paper Awards of the IEEE GC 2023, IEEE GC 2020, IEEE ICC 2021, ICCSPA 2019, IEEE WCNC 2016, IEEE ICC 2009, and the IEEE Canada Certificate of Appreciation in 2007 and 2008. He has been serving as the Co-Chair for the IoT and Sensor Networks Symposium of the GC22, the TPC Co-Chair of the ITC/ADC 2022 Conference, the Track Co-Chair for Antenna Systems, Propagation, and RF Design, IEEE Vehicular Technology Conference (VTC Fall'20), the Tutorial Chair for IEEE Canadian Conference in Electrical and Computer Engineering (CCECE 2020), the General Co-Chair, IEEE SmartNets 2019 Conference, the Co-Chair for the MAC and Cross Layer Design Track of the IEEE (WCNC) 2019 and the Wireless Communications Symposium of the IEEE ICC18 and the Ad-hoc, Sensor, and Mesh Networking Symposium of the IEEE GC17, the Technical Co-chair for the Fifth International Conference on Selected Topics in Mobile & Wireless Networking (MoWNet2016), the Track Co-Chair for Multiple Antenna and Cooperative Communications, IEEE Vehicular Technology Conference (VTC Fall'16), the Co-Chair for ACM Performance Evaluation of Wireless Ad Hoc, Sensor, and Ubiquitous Networks (ACMPE-WASUN'14) 2014, the Technical Co chair for the Wireless Networks Symposium, the 2012 Global Communications Conference, the Ad hoc, Sensor, and Mesh Networking Symposium of the 2010 ICC, and the 25th Queens Biennial Symposium on Communications. He also served as the Track Co-chair for the Radio Access Techniques of the 2006 IEEE VTC Fall 2006 and the Transmission Techniques of the IEEE VTC-Fall 2012. From September 2005 to November 2008, he was the Chair of the IEEE Montreal Chapter in Communications and Information Theory. He is an IEEE ComSoc Distinguished Lecturer. He served as an Associate Editor for the IEEE COMMUNICATIONS LETTERS, IEEE TRANSACTIONS ON SIGNAL PROCESSING, IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, *IET Wireless Sensor Systems*, IEEE WIRELESS COMMUNICATIONS LETTERS, *Transactions on Vehicular Technology*, and currently serves as an Editor for the IEEE TRANSACTIONS ON COMMUNICATIONS, IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, and IEEE INTERNET OF THINGS JOURNAL.